REPUBLIQUE DE DJIBOUTI Unité-Égalité-Paix
****

Ministère de l'Enseignement Supérieur et
De la recherche Université de Djibouti

THEME : Gestion de solidarité financière

Réaliser par :

Oumalkaire Hamid Mohamed

Rahma Hamoud Mahamoud

Rahma Guirreh Eleyeh

Mountaha Issa Aden

Encadrer par : **Dr. Moubarak barre**

2026/2027

Table des matières

INTRODUCTION GÉNÉRALE..... 5
CHAPITRE 1: ÉTUDE PRÉLIMINAIRE ET CADRE DU PROJET..... 6
1.1 Présentation du projet..... 6
1.2 Contexte et justification du marché de la location..... 6
1.3 Analyse de l'existant et étude comparative..... 7
1.4 Étude de faisabilité et cadre juridique..... 8
1.4.1 Statut d'éligibilité et conformité..... 8
CHAPITRE 2: ANALYSE ET SPÉCIFICATION DES BESOINS..... 9
2.1 Identification des acteurs..... 9
2.2 Analyse des besoins..... 9
2.2.1 Besoins fonctionnels..... 9
2.2.2 Besoins non fonctionnels..... 10
2.3 CAHIER DE CHARGE..... 11
2.3.1 Objectif du projet..... 11
2.3.2 Contraintes techniques et environnementales..... 11
2.3.3 Livrables attendus..... 11
2.4 Spécifications fonctionnelles détaillées..... 12
CHAPITRE 3 : Méthodologie et planification..... 13
3.1 Méthodologie Agile (Scrum)..... 13
3.2 Planification et organisation du travail..... 13
3.2.1 Décomposition des tâches (WBS)..... 14
3.2.2 Ordonnancement (Diagramme de Gantt, Diagramme de Pert, Diagramme de Ressource) 14
3.3 Gestion des risques et journal de bord..... 15
3.3.1 Identification et analyse des risques..... 15
3.3.2 Journal de bord..... 16
CHAPITRE 4 : Conception du système..... 18
4.1 Architecture logicielle..... 18
4.2 Modélisation UML..... 22
4.2.1 Diagramme de cas d'utilisation..... 22
4.2.2 Diagramme de classes..... 22
4.2.3 Diagramme d'objets..... 23
4.2.4 Diagrammes de séquence..... 24
4.2.5 Diagrammes d'états-transitions..... 24
4.2.6 Diagramme d'activité..... 25

4.2.7Diagramme collaboration.....26
CHAPITRE 5 : Implémentation et mise en œuvre.....28
5.1 Environnement de développement et prérequis.....28
5.2 Développement des modules critiques.....30
5.2.1 Interface utilisateur (UI/UX).....31
5.2.2 Back-end et base de données (MySQL).....38
5.3 Implémentation du Chat-bot intelligent.....41
CHAPITRE 6 : TESTS.....43
6.1 Stratégie de tests (Unitaires, Intégration, Système, Acceptation(recette)).....43
6.2 Évaluation des performances.....49
CHAPITRE 7 : Déploiement et Maintenance.....50
7.1 Déploiement.....50
7.1.1 Architecture de déploiement.....50
7.1.2 Option de déploiement (Déploiement local, Déploiement web).....51
7.1.3 Sécurisation du déploiement.....51
7.2 Maintenance.....52
7.2.1 Maintenance correction.....52
7.2.2 Maintenance évolutives.....52
7.2.3 Maintenance adaptative.....52
Conclusion.....53
BIBLIOGRAPHIE

Remerciements

Avant de présenter ce travail, nous tenons à exprimer notre profonde gratitude envers toutes les personnes qui ont contribué, de près ou de loin, à la réalisation de ce projet de gestion de solidarité financière.

Nous adressons nos remerciements les plus sincères et les plus respectueux à Monsieur le Professeur Moubarak barre, notre encadrant, pour son accompagnement bienveillant, ses précieux conseils et son suivi rigoureux tout au long de l'élaboration de ce projet. Sa disponibilité, son expertise et ses orientations éclairées ont été d'une aide inestimable pour mener à bien ce travail.

Nous remercions également l'ensemble du corps enseignant pour la qualité de la formation dispensée et les connaissances transmises, qui ont constitué le socle indispensable à la conception et au

développement de ce projet.

Enfin, nous exprimons notre reconnaissance à tous ceux qui, par leur soutien moral ou technique, ont contribué à l'aboutissement de ce travail.

INTRODUCTION

Dans un contexte où la gestion des ressources financières au sein des groupes et des communautés devient de plus en plus importante, la mise en place d'un système efficace et structuré est essentielle. La solidarité financière représente un moyen d'entraide permettant aux membres d'un groupe de faire face à des besoins financiers urgents ou planifiés.

Cependant, la gestion manuelle de ces contributions peut engendrer des problèmes tels que le manque de transparence, les erreurs de calcul, la mauvaise organisation des données et les difficultés de suivi. Afin de remédier à ces limitations, l'utilisation d'une solution informatique apparaît comme une alternative pertinente.

Le présent projet porte sur la conception et la réalisation d'un système de gestion de solidarité financière, permettant d'automatiser les différentes opérations liées à la collecte des cotisations, au suivi des transactions et à la distribution des aides. Ce système vise à garantir une meilleure organisation, une transparence accrue ainsi qu'une sécurisation des données.

Ainsi, la problématique principale de ce projet est la suivante :

Comment concevoir et développer un système de gestion de solidarité financière fiable, sécurisé et efficace, répondant aux besoins des utilisateurs ?

Dans ce cadre, ce rapport est structuré en trois grandes parties. La première partie sera consacrée à l'étude du projet, incluant le contexte, les objectifs, le périmètre, ainsi que l'identification des besoins et des contraintes. La deuxième partie portera sur la conception du système à travers les différents diagrammes UML, notamment les vues statiques et dynamiques. Enfin, la dernière partie présentera la réalisation du système, les technologies utilisées, les tests effectués ainsi que la validation finale de la solution.

CHAPITRE 1 : ÉTUDE PRÉLIMINAIRE ET CADRE DU PROJET

1.1 Présentation du projet

Le présent projet consiste à concevoir et développer **un système informatisé de gestion de solidarité financière.**
Ce système vise à faciliter l'organisation, la gestion et le suivi des contributions financières entre les membres d'un groupe solidaire.

Dans de nombreuses communautés, la solidarité financière repose sur des systèmes informels où les membres contribuent régulièrement à une caisse commune pour aider les participants en cas de besoin (maladie, mariage, urgence, etc.). Cependant, ces systèmes présentent souvent des limites en termes de transparence, de sécurité et de gestion.

Le système proposé, développé avec le Framework **Laravel 12**, permettra de digitaliser ces processus en offrant une plateforme centralisée, sécurisée et facile à utiliser.

Trois acteurs principaux interviennent dans le système :

- **Membre** : participe aux cotisations et peut faire des demandes d'aide
- **Caissière** : gère les entrées et sorties d'argent
- **Administrateur** : supervise et contrôle l'ensemble du système

1.2 Contexte et justification du marché de la location

Dans plusieurs pays, notamment en Afrique (y compris à Djibouti), les systèmes de solidarité financière sont très répandus sous forme de :

- tontines
- associations d'entraide
- caisses communautaires

Cependant, la gestion de ces systèmes est souvent :

- manuelle (cahiers, Excel, etc.)
- sujette aux erreurs humaines
- peu transparente
- difficile à contrôler

Avec l'évolution du numérique, il devient nécessaire de moderniser ces pratiques afin de :

- améliorer la transparence financière
- sécuriser les transactions
- faciliter la gestion des membres
- assurer une traçabilité complète

-Ainsi, ce projet répond à un besoin réel du marché :
digitaliser la gestion des systèmes de solidarité financière.

1.3 Analyse de l'existant et étude comparative

-Analyse de l’existant

Actuellement, les systèmes de solidarité financière fonctionnent principalement de deux manières :

1. Gestion manuelle

- Utilisation de cahiers pour enregistrer les cotisations
- Calculs faits à la main
- Risque élevé d’erreurs
- Difficulté de suivi

2. Utilisation d’outils basiques

- Excel ou fichiers simples
- Manque d’automatisation
- Pas de gestion des rôles
- Sécurité limitée

-Étude comparative

Critère	Système manuel	Excel	Système proposé
Sécurité	**Faible**	Moyenne	**Élevée**
Transparence	**Faible**	Moyenne	**Élevée**
Automatisation	**Faible**	**Partielle**	**Élevée**
Gestion multi-utilisateurs	**Faible**	**Moyenne**	**Élevée**
Traçabilité	**Faible**	**Faible**	**Élevée**

Le système proposé apporte une **amélioration significative** en termes de :

- Fiabilité
- Sécurité
- Efficacité

1.4 Étude de faisabilité et cadre juridique

1.4.1 Statut d'éligibilité et conformité

Le système doit respecter certaines conditions pour être utilisable :

✔ Être accessible aux membres inscrits uniquement
✔ Garantir l'authentification sécurisée des utilisateurs
✔ Respecter les règles de gestion financière de l'association
✔ Assurer la confidentialité des données

En termes de conformité :

- Respect des lois locales sur les données personnelles
- Respect des règles de gestion associative
- Traçabilité des opérations financières

1.4.2 Gestion des risques légaux

L'utilisation d'un système de gestion financière comporte plusieurs risques juridiques qu'il faut anticiper :

1.Risques identifiés

- Mauvaise gestion des fonds
- Accès non autorisé aux données
- Fraudes ou falsifications
- Perte de données

2.Mesures de prévention

Pour réduire ces risques, le système intègre :

✔ Authentification sécurisée (login/mot de passe)
✔ Gestion des rôles (Membre, Caissière, Admin)
✔ Journalisation des actions (logs)
✔ Sauvegarde régulière des données
✔ Contrôle des accès

CHAPITRE 2 : ANALYSE ET SPÉCIFICATION DES BESOINS

2.1 Identification des acteurs

Le système de gestion de solidarité financière implique trois acteurs principaux qui interagissent avec la plateforme :

-Membre

Le membre est un utilisateur inscrit dans le système.
Il participe activement à la solidarité financière.

Rôles **:**

- S'inscrire et se connecter
- Consulter ses cotisations
- Effectuer des paiements
- Faire une demande d'aide
- Consulter l'historique

-Caissière

La caissière est responsable de la gestion financière quotidienne.

Rôles :

- Enregistrer les cotisations
- Valider les paiements
- Gérer les décaissements
- Consulter les rapports financiers

-Administrateur

L'administrateur supervise l'ensemble du système.

Rôles :

- Gérer les utilisateurs
- Attribuer les rôles
- Configurer le système
- Contrôler les activités
- Générer les rapports globaux

2.2.1 Besoins fonctionnels

Les besoins fonctionnels décrivent les fonctionnalités que le système doit offrir.

- Gestion des utilisateurs

- Inscription des membres

- Authentification (login/logout)
- Gestion des rôles (Admin, Caissière, Membre)

- Gestion des cotisations

- Enregistrement des cotisations
- Suivi des paiements
- Historique des transactions

- Gestion des aides financières

- Demande d'aide par un membre
- Validation par l'administrateur
- Décaissement par la caissière

-Gestion des finances

- Suivi des entrées et sorties
- Calcul automatique du solde
- Génération de rapports

-Notifications (optionnel avancé)

- Alertes de paiement
- Notifications de validation

2.2.2 Besoins non fonctionnels

Les besoins non fonctionnels définissent les contraintes de qualité du système.

-Sécurité

- Authentification sécurisée
- Protection des données
- Gestion des accès

-Performance

- Temps de réponse rapide
- Gestion efficace des données

-Fiabilité

- Système stable
- Sauvegarde régulière

-Ergonomie

- Interface simple et intuitive
- Facilité d'utilisation

- Accessibilité

- Accessible via navigateur web
- Compatible avec différents appareils

2.3 Cahier de charge

2.3.1 Objectif du projet

L'objectif principal du projet est de développer un système permettant :

✔ La gestion automatisée des cotisations
✔ La transparence financière
✔ La gestion des membres
✔ La sécurisation des transactions
✔ L'amélioration de l'organisation

2.3.2 Contraintes techniques et environnementales

-Contraintes techniques

- Utilisation de **PHP avec Laravel 12**
- Base de données (MySQL ou PostgreSQL)
- Architecture MVC
- Hébergement web requis

-Contraintes matérielles

- Ordinateur pour l'administration
- Accès Internet

- Contraintes environnementales

- Adaptation au contexte local (Djibouti)
- Utilisation dans des zones avec connexion limitée

2.3.3 Livrables attendus

À la fin du projet, les éléments suivants doivent être livrés :

-Document de rapport (50 pages)
-Diagrammes UML (cas d'utilisation, classes, séquence…)
- Application web fonctionnelle
- Code source (Laravel 12)
- Base de données
-Présentation PowerPoint

2.4 Spécifications fonctionnelles détaillées

Cette section décrit en détail les fonctionnalités du système.

- Authentification

- L'utilisateur entre son email et mot de passe
- Vérification dans la base de données
- Redirection selon le rôle

-Gestion des membres

- Ajouter un membre
- Modifier les informations
- Supprimer un membre
- Lister tous les membres

- Gestion des cotisations

- Enregistrer une cotisation
- Associer à un membre
- Mettre à jour le statut (payé/non payé)

-Gestion des aides

- Soumettre une demande
- Examiner la demande
- Approuver ou refuser
- Effectuer le paiement

-Gestion des rapports

.Rapport des cotisations

. Rapport des dépenses

. Statistiques globales

-Scénario simplifié du système

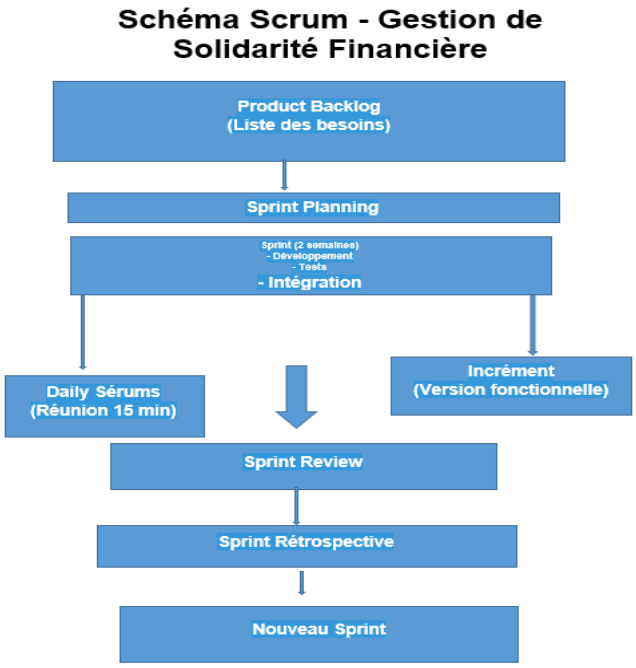
1. Le membre s'inscrit
2. Il paie sa cotisation
3. La caissière enregistre le paiement
4. Le membre fait une demande d'aide
5. L'administrateur valide
6. La caissière effectue le paiement

CHAPITRE 3 : MÉTHODOLOGIE ET PLANIFICATION

3.1 Méthodologie Agile (Scrum)

Dans le cadre de ce projet, nous avons adopté la méthodologie **Agile Scrum**, qui permet une gestion flexible et progressive du développement.

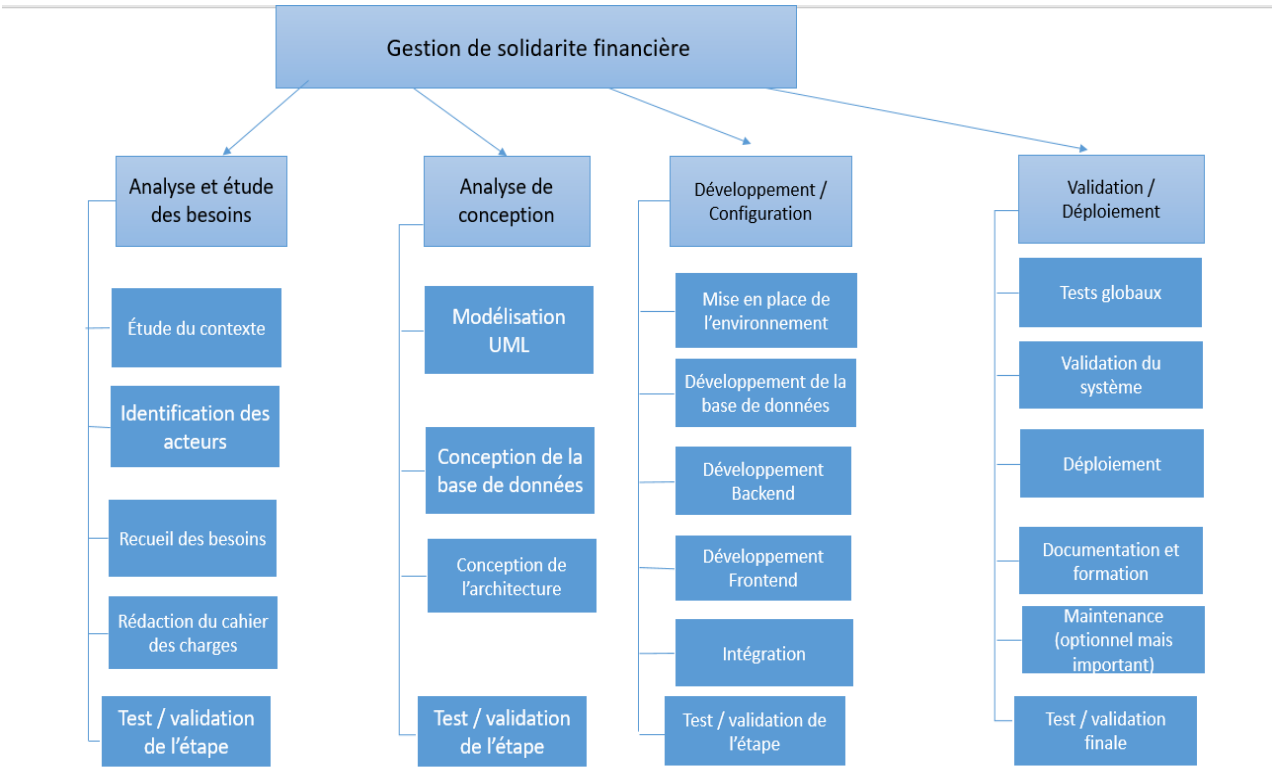
Scrum repose sur des cycles courts appelés **sprints**, permettant de livrer régulièrement des fonctionnalités opérationnelles.



3.2 Planification et organisation du travail

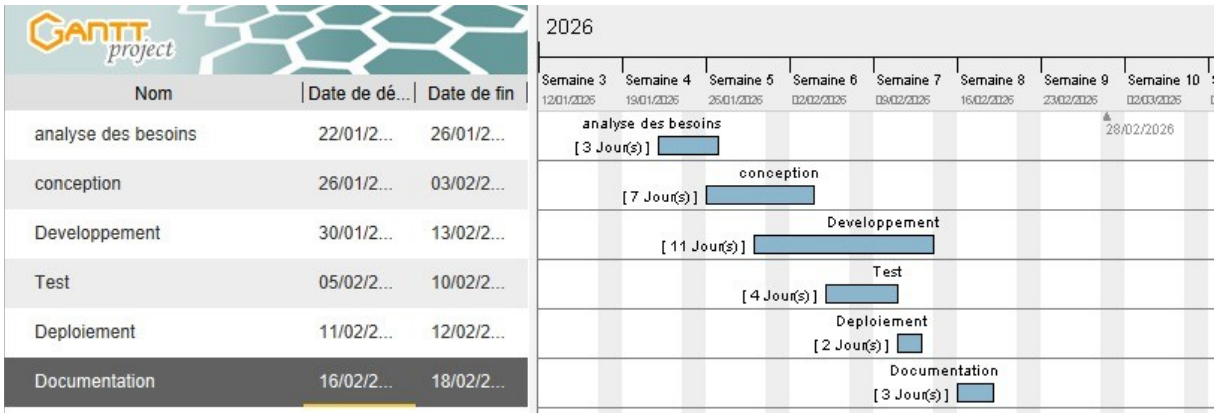
3.2.1 Décomposition des tâches (WBS)

Le WBS (Work Breakdown Structure) permet de découper le projet en tâches organisées.



-Diagramme de Gantt

Le diagramme de Gantt permet de visualiser l'enchaînement des tâches nécessaires au développement du système de gestion de solidarité financière. Chaque activité y est représentée par une barre horizontale dont la longueur correspond à sa durée en jours. Les relations de dépendance entre les tâches permettent d'identifier l'ordre exact d'exécution et d'assurer une planification cohérente.



-Diagramme de perte

3.3 Gestion des risques et journal de bord

3.3.1 Identification et analyse des risques

- Gestion des risques

- **Risque technique**

Les risques techniques concernent les erreurs de calcul des cotisations, la perte ou la corruption des données, les pannes du système ainsi que les problèmes de sécurité et d'accès non autorisé. Pour réduire ces risques, il est nécessaire de mettre en place des sauvegardes régulières, de sécuriser l'accès au système, de réaliser des tests fréquents et d'assurer la maintenance continue de l'application.

- **4.2 risque d'organisation**

Les risques organisationnels sont liés à la gestion du projet et au fonctionnement du groupe, notamment la mauvaise gestion des cotisations, l'absence de règles claires, le manque de coordination et les retards dans l'avancement. Pour y faire face, il est important de définir des règles de gestion précises, de planifier les tâches, de répartir les responsabilités et de suivre régulièrement les activités.

- **4.3 risque humains**

Les risques humains concernent les erreurs de saisie, la mauvaise utilisation du système, le manque de formation et les conflits entre les membres. Afin de limiter ces risques, il est recommandé de former les utilisateurs, de concevoir une interface simple, de mettre en place des contrôles de validation et de renforcer la communication entre les membres.

- ❖ Tout projet informatique comporte des risques qu'il faut anticiper et gérer de manière proactive. Le tableau suivant identifie les principaux risques associés au projet SolidariFin, leur niveau d'impact et les mesures de mitigation correspondantes.

⚠ Risque	Impact	Solution / Mitigation
Retard de développement	**Élevé**	Bonne planification

⚠ Risque	Impact	Solution / Mitigation
Bugs techniques	**Moyen**	Tests réguliers
Perte de données	**Élevé**	Sauvegardes régulières
Mauvaise compréhension des besoins	**Élevé**	Validation continue avec les utilisateurs
Problème de connexion réseau	**Moyen**	Mode hors-ligne / mode simple

Légende : **Élevé** **Moyen**

-Stratégie de gestion

- Planification Agile
- Tests fréquents
- Communication régulière
- Sauvegardes automatiques

3.3.2 Journal de bord

Le journal de bord retrace les principales étapes et activités réalisées tout au long du projet. Il permet de suivre l'avancement, les décisions prises et les difficultés rencontrées.

Semaine	Activités réalisées	Observations
Semaine 1	Définition du sujet, recherche documentaire, analyse de l'existant	Choix du thème validé
Semaine 2	Rédaction du cahier des charges, identification des besoins fonctionnels et non fonctionnels	Besoins bien identifiés
Semaine 3	Modélisation UML : diagrammes de cas d'utilisation, diagramme de classes	Validation des diagrammes
Semaine 4	Modélisation UML : diagrammes de séquence,	Conception avancée terminée

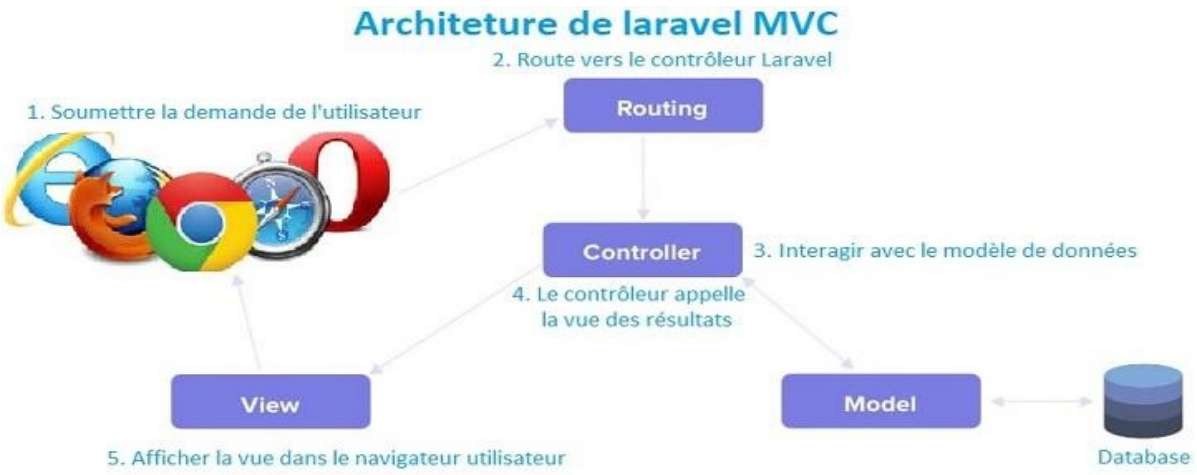
	d'activité, d'états-transitions	
Semaine 5	Installation de l'environnement Laravel 12, création de la base de données MySQL	Environnement opérationnel
Semaine 6	Développement des modules : authentification, gestion des membres	Modules fonctionnels
Semaine 7	Développement des modules : cotisations, demandes d'aide, tableau de bord	Intégration en cours
Semaine 8	Tests unitaires et d'intégration, correction des bugs	Tests validés
Semaine 9	Finalisation de l'interface utilisateur, tests d'acceptation	Interface validée
Semaine 10	Rédaction du rapport, préparation de la soutenance	Rapport finalisé

Ce journal de bord montre une progression régulière et méthodique du projet, depuis la phase d'analyse jusqu'à la finalisation du rapport.

CHAPITRE 4 : CONCEPTION DU SYSTÈME

4.1 Architecture logicielle

Le système est basé sur une architecture **MVC (Modèle – Vue – Contrôleur)**, intégrée dans le framework Laravel.



-Composants

- **Modèle (Model)** : gère la base de données (Membre, Cotisation, Aide…)
- **Vue (View)** : interface utilisateur (Blade Laravel)
- **Contrôleur (Controller)** : logique métier

- Architecture technique utilisées

Frontend (interface utilisateur)

La partie frontend de l'application a été réalisée à l'aide des technologies web standards. Le langage HTML5 a été utilisé pour structurer les différentes pages de l'application, en définissant les éléments tels que les formulaires, les tableaux et les interfaces d'affichage.

Le langage CSS3 a été utilisé pour la mise en forme et l'amélioration de l'aspect visuel de l'application. Il permet de gérer la disposition des éléments, les couleurs, les polices et l'adaptabilité de l'interface.

Le langage JavaScript a été intégré afin d'ajouter de l'interactivité au système.

Backend (logique métier)

La partie backend assure le traitement des données et la gestion de la logique métier de l'application. Elle a été développée en utilisant le framework Laravel, basé sur le langage PHP.

Ce composant permet de traiter les requêtes envoyées par les utilisateurs, de gérer l'authentification, d'effectuer les calculs financiers et d'assurer la communication avec la base de données.

Laravel facilite également le développement grâce à :

une organisation structurée suivant l'architecture MVC (Modèle – Vue – Contrôleur)

un système de routage performant

des outils de sécurité intégrés (protection CSRF, validation des données)

l'ORM Eloquent pour simplifier les opérations sur la base de données

Base de données

La gestion des données a été assurée par le système de gestion de base de données relationnelle MySQL.

Cette base de données permet de stocker de manière structurée les informations relatives aux utilisateurs, aux cotisations et aux transactions financières. Elle garantit également l'intégrité des données grâce à l'utilisation de clés primaires et étrangères, ainsi que la cohérence des relations entre les différentes tables.

Dans Laravel, la gestion de la base de données est facilitée grâce aux migrations, qui permettent de créer et de modifier les tables de manière organisée et versionnée.

Outils complémentaires

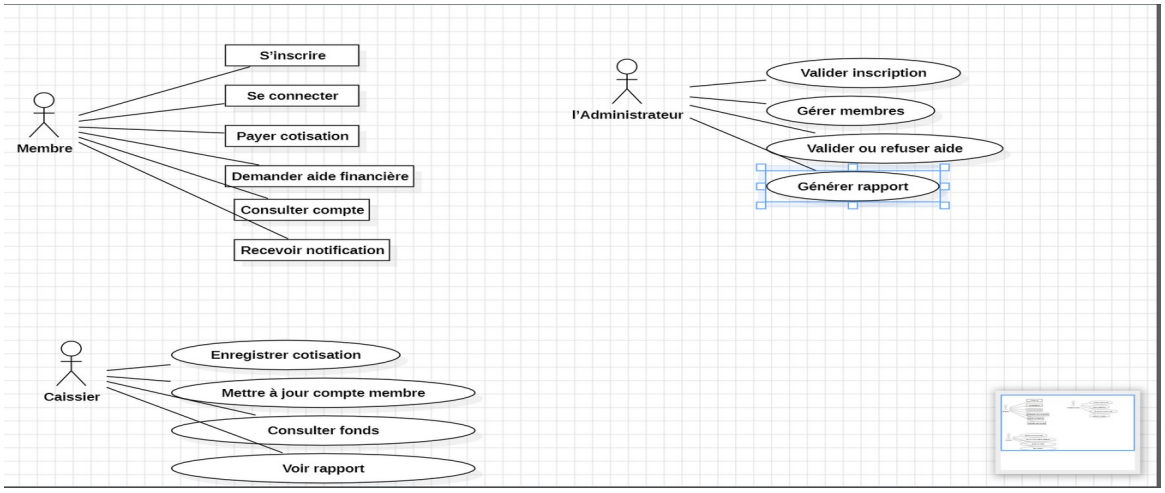
Dans le cadre du développement, nous avons utilisé Git comme outil de gestion de version. Il permet de suivre les modifications du code, de gérer les différentes versions du projet et de faciliter le travail collaboratif.

Enfin, un serveur local tel que XAMPP ou WAMP a été utilisé pour exécuter l'application en local. Cet environnement permet de simuler un serveur web en intégrant les composants nécessaires tels que le serveur Apache, le langage PHP et le système de gestion de base de données.

4.2 Modélisation UML

4.2.1 Diagramme de cas d'utilisation

Le diagramme de cas d'utilisation présente les interactions entre les acteurs (Membre, Caissière, Administrateur) et le système.
Il permet de visualiser les principales fonctionnalités comme la gestion des cotisations, des aides et des utilisateurs.
Il donne une vue globale du fonctionnement du système.



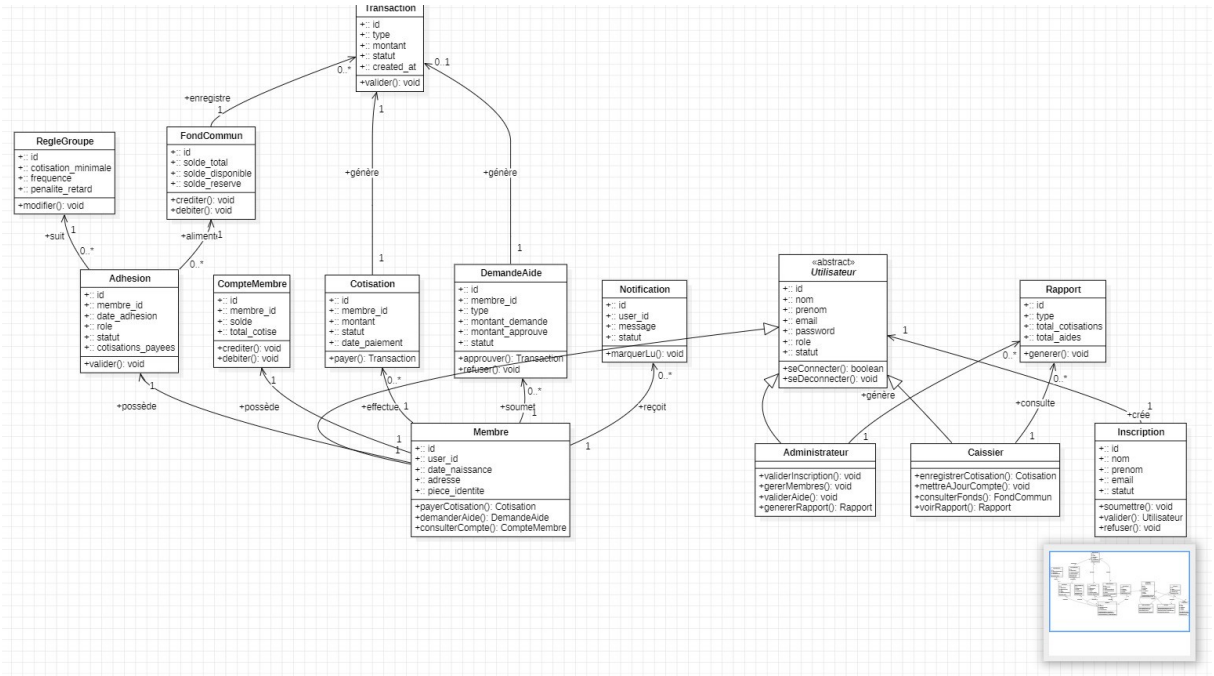
4.2.2 Un diagramme de classes

Un diagramme de classes est un schéma UML qui sert à représenter la structure d'un système informatique. Il montre les classes (objets principaux du système), leurs attributs (données), leurs méthodes (fonctions) ainsi que les relations entre elles. En clair : c'est le plan de

construction du logiciel, qui permet de visualiser l'organisation du système avant sa réalisation.

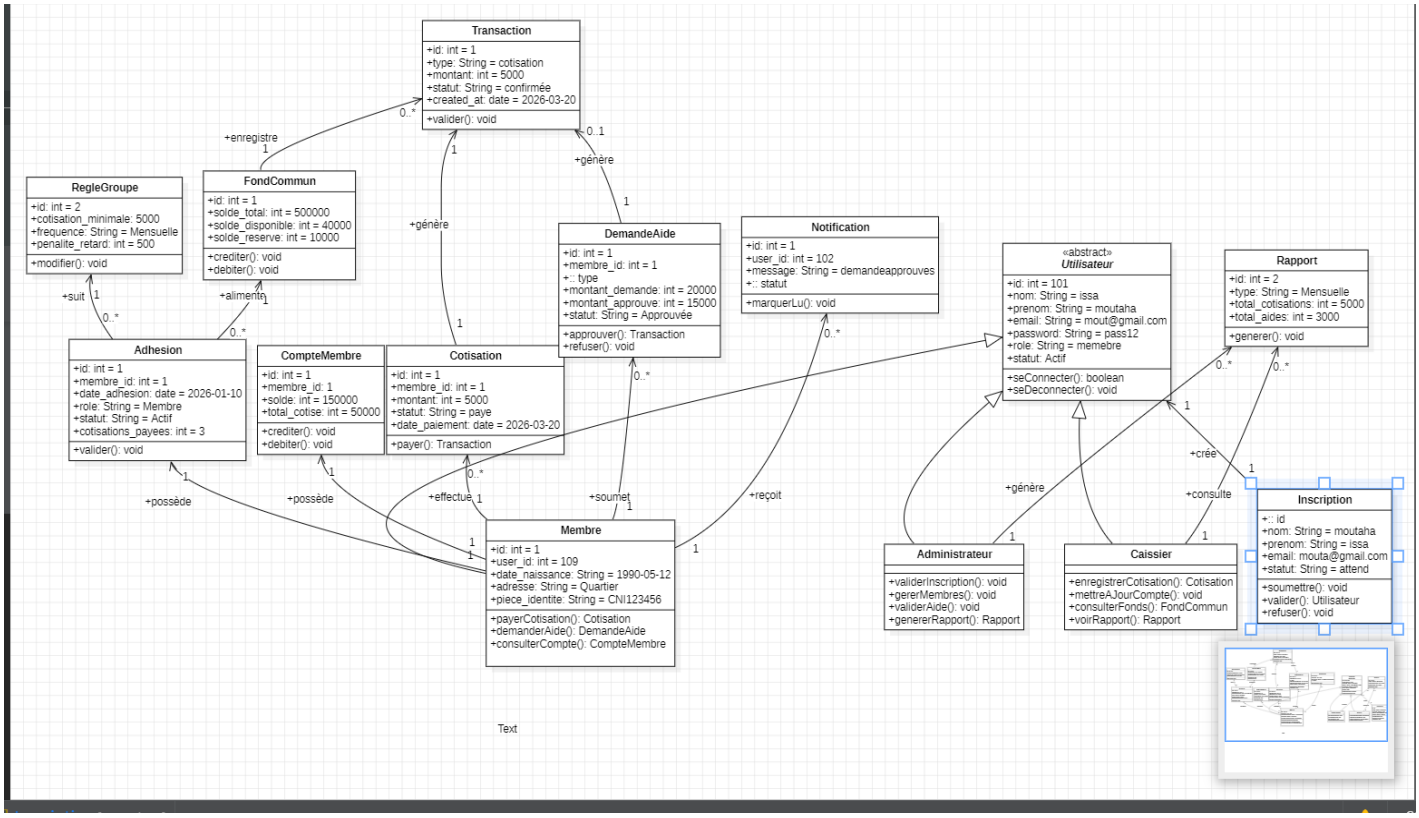
- Le diagramme de classes de notre projet de gestion de solidarité financière est composé de plusieurs classes principales telles que Utilisateur, Administrateur, Caissier, Membre, Cotisation, DemandeAide, Transaction, Compte Membre, FondCommun et Rapport. Il permet de représenter de manière claire la structure du système, les données manipulées ainsi que les relations entre les différentes entités.

Diagramme de classes (Modèle de données)



4.2.3 Diagramme d'objets

Le diagramme d’objets représente des instances réelles du système.



4.2.4 Diagrammes de séquence

-Processus : Demande d’aide

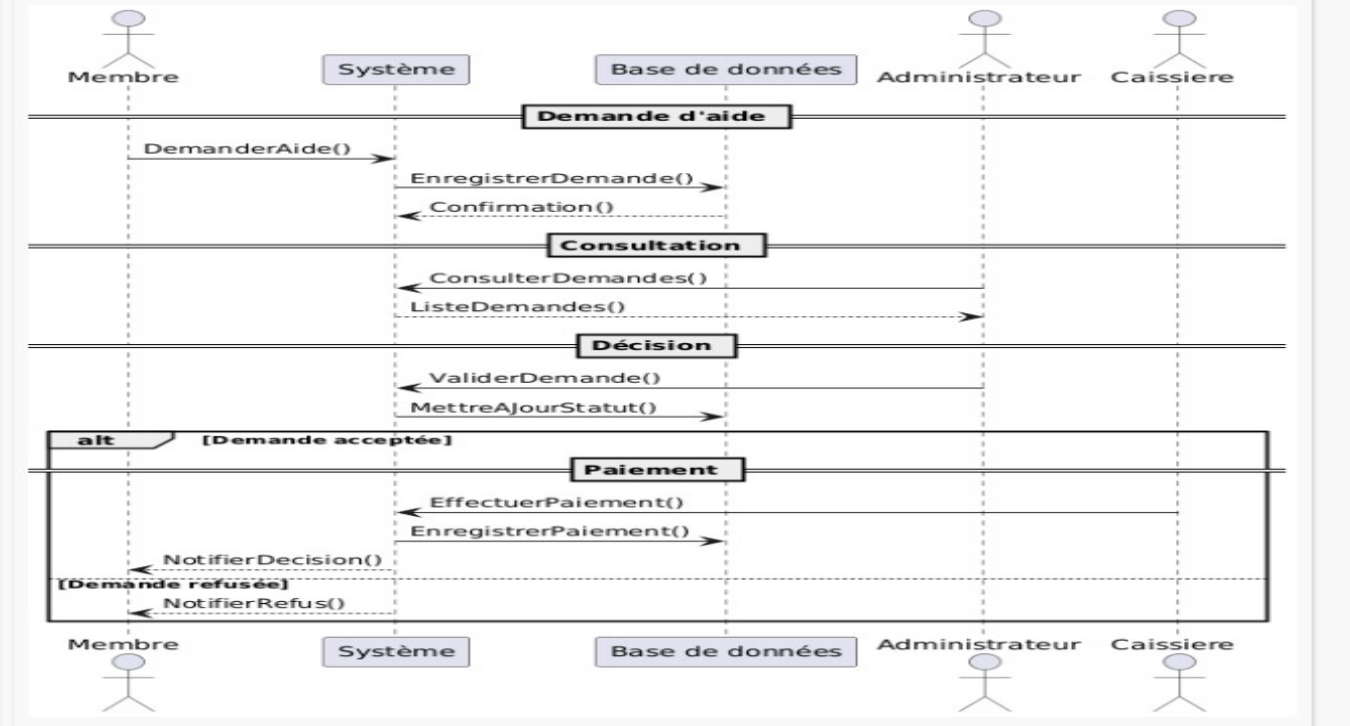
Ce diagramme montre l’ordre des interactions dans le temps.

Demande d'aide : Le membre soumet une demande d'aide ; le système l'enregistre et confirme la réception.

Consultation : L’administrateur consulte la liste des demandes enregistrées dans la base de données.

Décision : L’administrateur valide la demande et le système met à jour son statut.

Paiement : *Si acceptée :* La caissière effectue le paiement et le système l'enregistre ; le membre est notifié. *Si refusée :* Le membre reçoit une notification de refus.

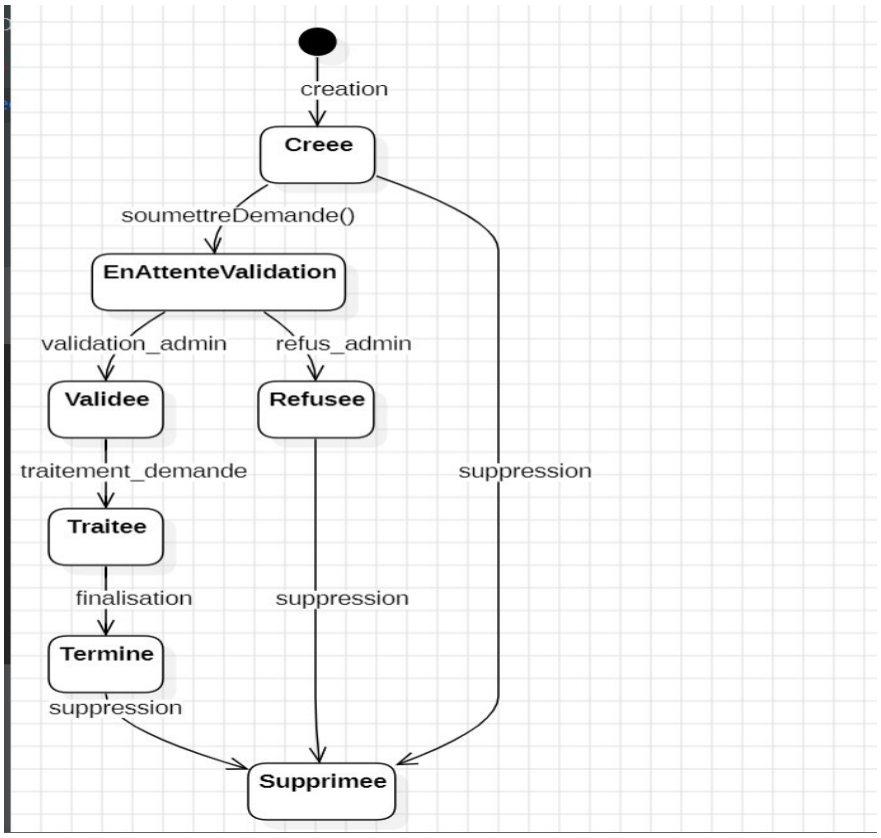


4.2.5 Diagrammes d'états-transitions

Un diagramme d'état est un diagramme UML qui représente les différents états d'un objet ou d'un système ainsi que les transitions entre ces états en fonction d'événements. Il montre comment un élément évolue dans le temps selon les actions ou conditions qui se produisent.

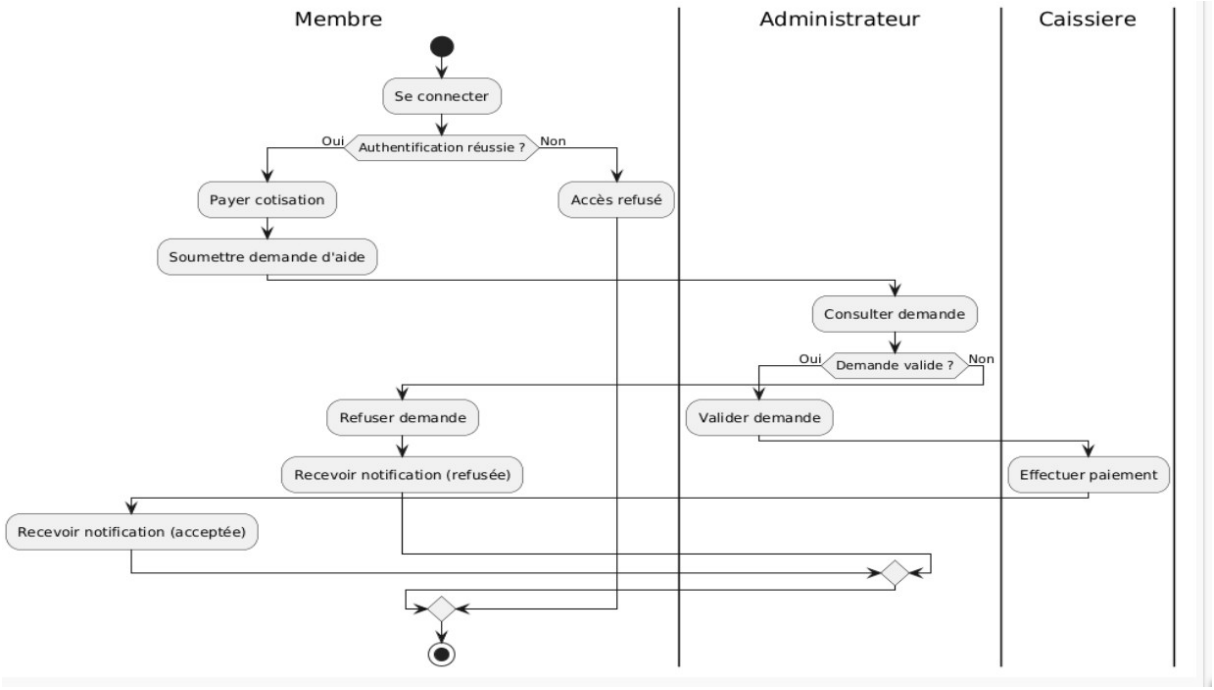
Le diagramme d'état sert à :

- Comprendre le comportement dynamique d'un système ou d'un objet.
- Visualiser les changements d'état (exemple : en attente → en cours → terminé).
- Décrire les réactions du système lorsqu'un événement se produit.
- Éviter les erreurs de logique dans le fonctionnement du système.
- Faciliter la conception et la communication entre les membres d'un projet.



4.2.6 Diagramme d’activité

Ce diagramme d'activité illustre le flux complet de traitement d'une demande d'aide au sein du système SolidariFin. Le processus débute par la connexion du membre ; en cas d'authentification réussie, il peut payer sa cotisation et soumettre une demande. L'administrateur consulte ensuite cette demande et prend une décision : si elle est validée, la caissière procède au paiement et le membre reçoit une notification d'acceptation ; dans le cas contraire, le membre est notifié du refus. Le processus se termine après la convergence des deux branches.



4.2.7Diagramme collaboration

Le diagramme de collaboration représente les communications entre les objets du système (**QUI communique avec QUI).** Il met en évidence les relations entre les composants impliqués dans un scénario.

Ce qui est important :

✔ objets reliés
✔ messages numérotés

1. Acteurs principaux

- **Membre** → fait des demandes (cotisation, aide)
- **Administrateur** → valide et gère le système

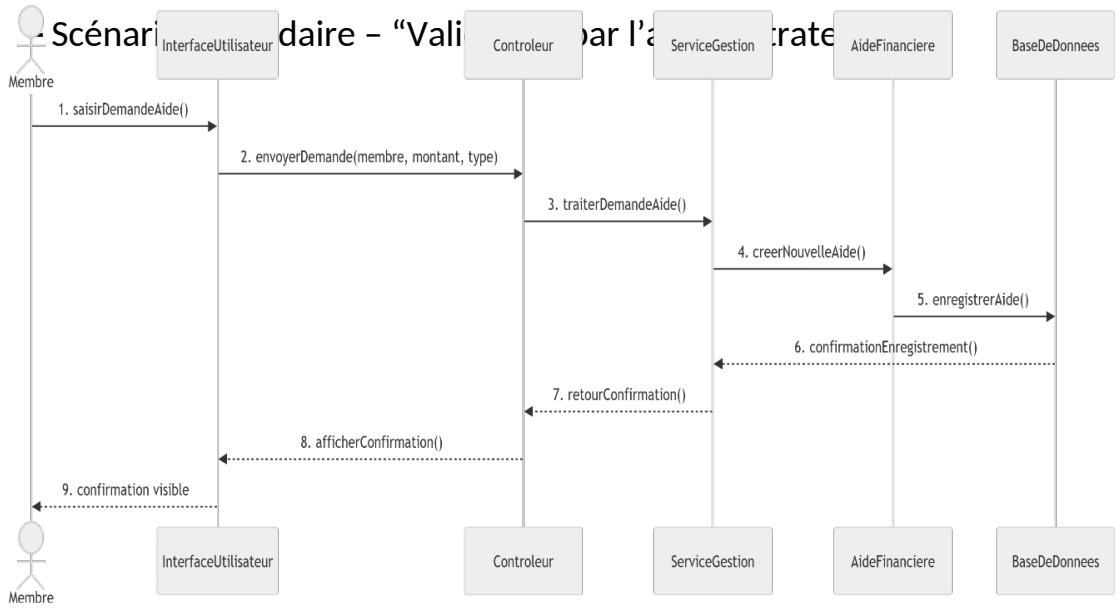
2. Objets du système

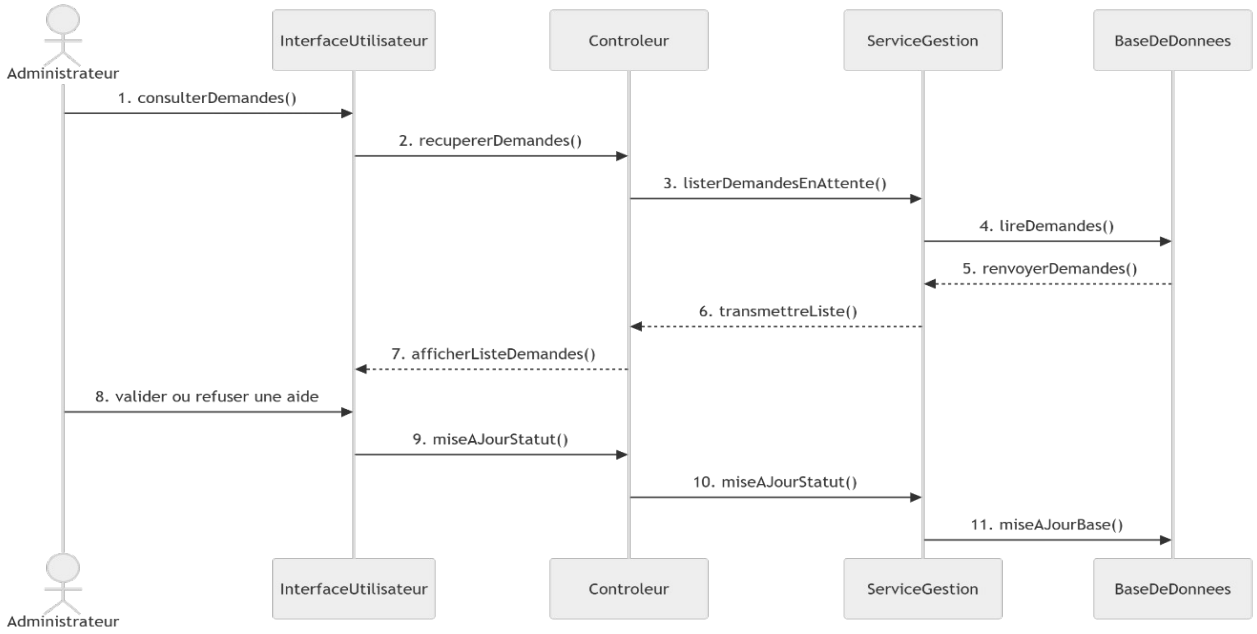
- **InterfaceUtilisateur** → interface front-end (formulaires, tableaux)
- **Controleur** → gère les interactions et transfère les requêtes au service
- **ServiceGestion** → logique métier (traitement des demandes, règles)
- **Entités Métier** → `Membre`, `Cotisation`, `AideFinanciere`, `Remboursement`

  BaseDeDonnees → stockage persistant

-Scénario principal – “Demande d’aide financière”

- Scénari daire – “Vali par l’a trate





CHAPITRE 5 : IMPLÉMENTATION ET MISE EN ŒUVRE

5.1 Environnement de développement et prérequis

Avant de commencer le développement de l'application de gestion de solidarité financière, il est nécessaire de mettre en place un environnement de travail adapté. Cette section présente les outils, logiciels et technologies utilisés pour la réalisation du projet.

Outils et logiciels utilisés

Outil / Technologie	Rôle	Version
PHP	Langage de programmation back-end	8.2
Laravel	Framework PHP (architecture MVC)	12
MySQL	Système de gestion de base de données	8.0
Composer	Gestionnaire de dépendances PHP	2.x
WampServer	Serveur local (Apache + MySQL + PHP)	8.2
Visual Studio Code	Éditeur de code source	Dernière version
Git	Système de contrôle de version	2.x
Blade	Moteur de templates Laravel	Intégré à Laravel
Bootstrap	Framework CSS pour le design responsive	5.3
StarUML	Outil de modélisation UML	5.x
Navigateur web	Test et visualisation de l'application	Chrome / Firefox

Prérequis d'installation

Pour faire fonctionner le projet en local, les prérequis suivants doivent être respectés :

- PHP version 8.2 ou supérieure installé sur la machine ;
- Composer installé pour gérer les dépendances Laravel ;
- WampServer ou un serveur similaire pour Apache et MySQL ;
- Node.js et NPM pour la compilation des assets front-end ;
- Un navigateur web moderne (Google Chrome, Mozilla Firefox) ;
- Connexion Internet pour l'installation initiale des packages.

Étapes d'installation du projet

L'installation du projet suit les étapes suivantes :

- 1. Installer WampServer et démarrer les services Apache et MySQL ;
- 2. Installer Composer sur la machine ;
- 3. Créer un nouveau projet Laravel avec la commande : composer create-project laravel/laravel solidarite-financiere ;
- 4. Configurer le fichier .env avec les paramètres de la base de données (DB_DATABASE, DB_USERNAME, DB_PASSWORD) ;
- 5. Créer la base de données dans phpMyAdmin ;
- 6. Exécuter les migrations avec la commande : php artisan migrate ;
- 7. Lancer le serveur de développement avec : php artisan serve ;
- 8. Accéder à l'application via le navigateur à l'adresse : http://127.0.0.1:8000

5.2 Développement des modules critiques

Le développement de l'application a été réalisé de manière modulaire, en suivant l'architecture MVC (Modèle-Vue-Contrôleur) proposée par Laravel. Chaque fonctionnalité du système correspond à un module spécifique qui a été développé, testé, puis intégré progressivement.

Les modules critiques développés dans cette application sont les suivants :

Module	Description	Acteurs concernés
Authentification	Inscription, connexion, déconnexion et gestion des sessions	Tous les acteurs
Gestion des membres	Ajout, modification, suppression et liste des membres	Administrateur
Gestion des cotisations	Enregistrement, suivi et historique des cotisations	Caissière, Membre
Gestion des aides	Demande, validation, refus et	Membre, Admin, Caissière

	paiement des aides financières	
Tableau de bord	Vue globale avec statistiques et résumé financier	Administrateur, Caissière
Gestion des rapports	Génération de rapports de cotisations, dépenses et statistiques	Administrateur

5.2.1 Interface utilisateur (UI/UX)

L'interface utilisateur a été conçue avec le souci de proposer une expérience simple, intuitive et agréable pour tous les acteurs du système, y compris les utilisateurs qui ne sont pas habitués aux outils informatiques.

Technologies utilisées pour le front-end

- Blade : le moteur de templates intégré à Laravel, permettant de créer des vues dynamiques avec une syntaxe simple ;
- Bootstrap 5 : framework CSS utilisé pour le design responsive et les composants visuels (boutons, formulaires, tableaux, cartes) ;
- HTML5 et CSS3 : pour la structure et la mise en forme des pages ;
- JavaScript : pour les interactions dynamiques côté client (alertes, confirmations, filtres).

Principes de conception UI/UX appliqués

Plusieurs principes ont été respectés lors de la conception de l'interface :

- Simplicité : les menus et formulaires sont clairs, avec un minimum de champs pour éviter la confusion ;
- Cohérence : les couleurs, polices et boutons sont uniformes sur toutes les pages ;
- Navigation fluide : un menu latéral (sidebar) permet d'accéder rapidement aux différentes sections ;
- Feedback utilisateur : des messages de confirmation ou d'erreur s'affichent après chaque action (ajout, modification, suppression) ;
- Responsive design : l'interface s'adapte aux écrans d'ordinateur, de tablette et de téléphone.

Pages principales de l'application

Page	Description
Page de connexion	Formulaire d'authentification avec email et mot de passe
Tableau de bord	Vue d'ensemble avec les statistiques (nombre de membres, total cotisations, solde)
Liste des membres	Affichage sous forme de tableau avec options d'ajout, modification et suppression
Formulaire d'ajout de membre	Formulaire avec champs : nom, prénom, téléphone, adresse, date d'inscription
Liste des cotisations	Historique des cotisations avec filtre par membre et par date
Formulaire de cotisation	Enregistrement d'une cotisation avec sélection du membre et du montant
Demandes d'aide	Liste des demandes avec statut (en attente, approuvée, refusée)
Rapports financiers	Résumé des entrées, sorties et solde global

Captures d'écran de l'application

Les figures suivantes présentent les principales interfaces de l'application de gestion de solidarité financière :

Figure 1 : Page de connexion (Login)

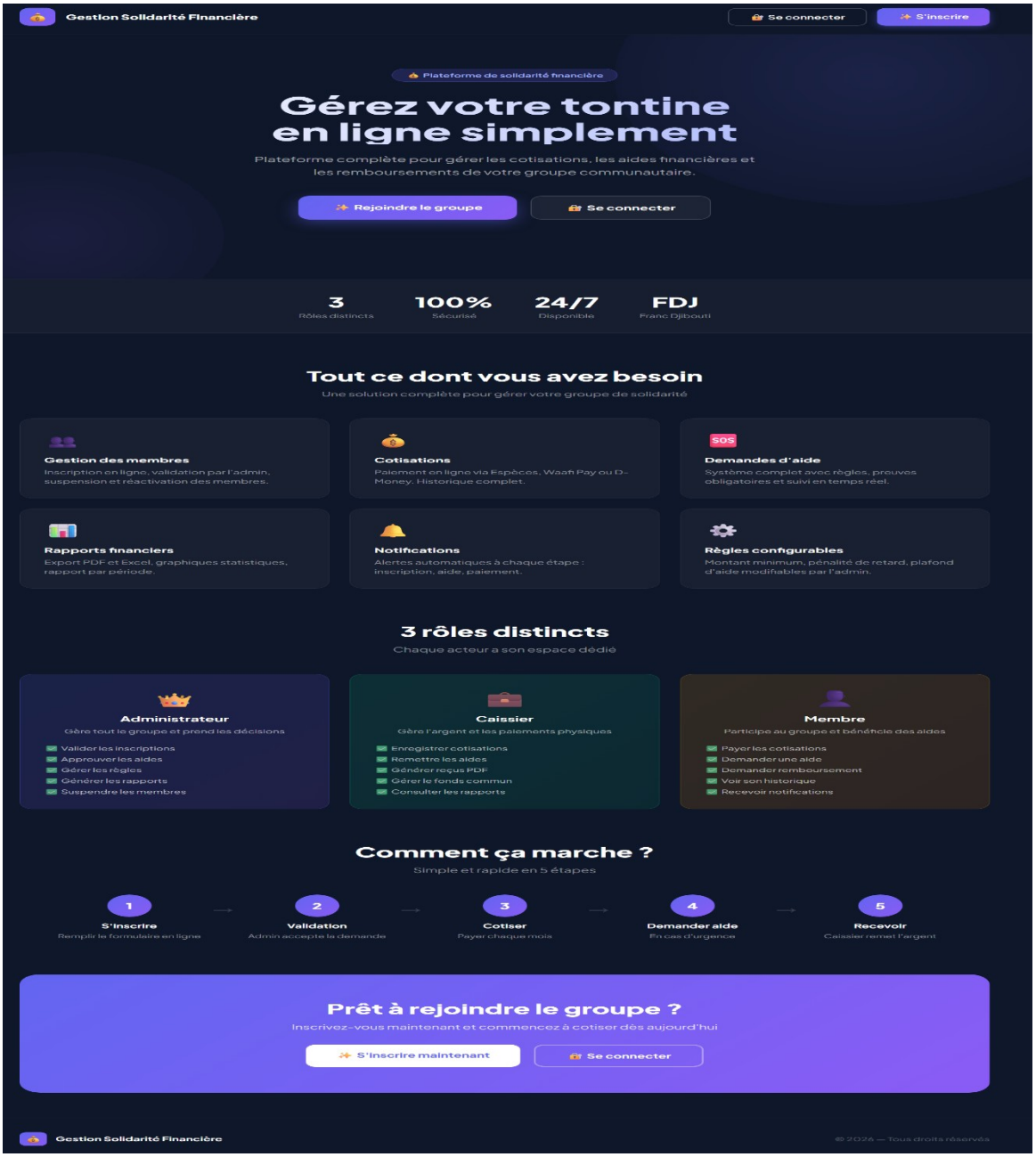
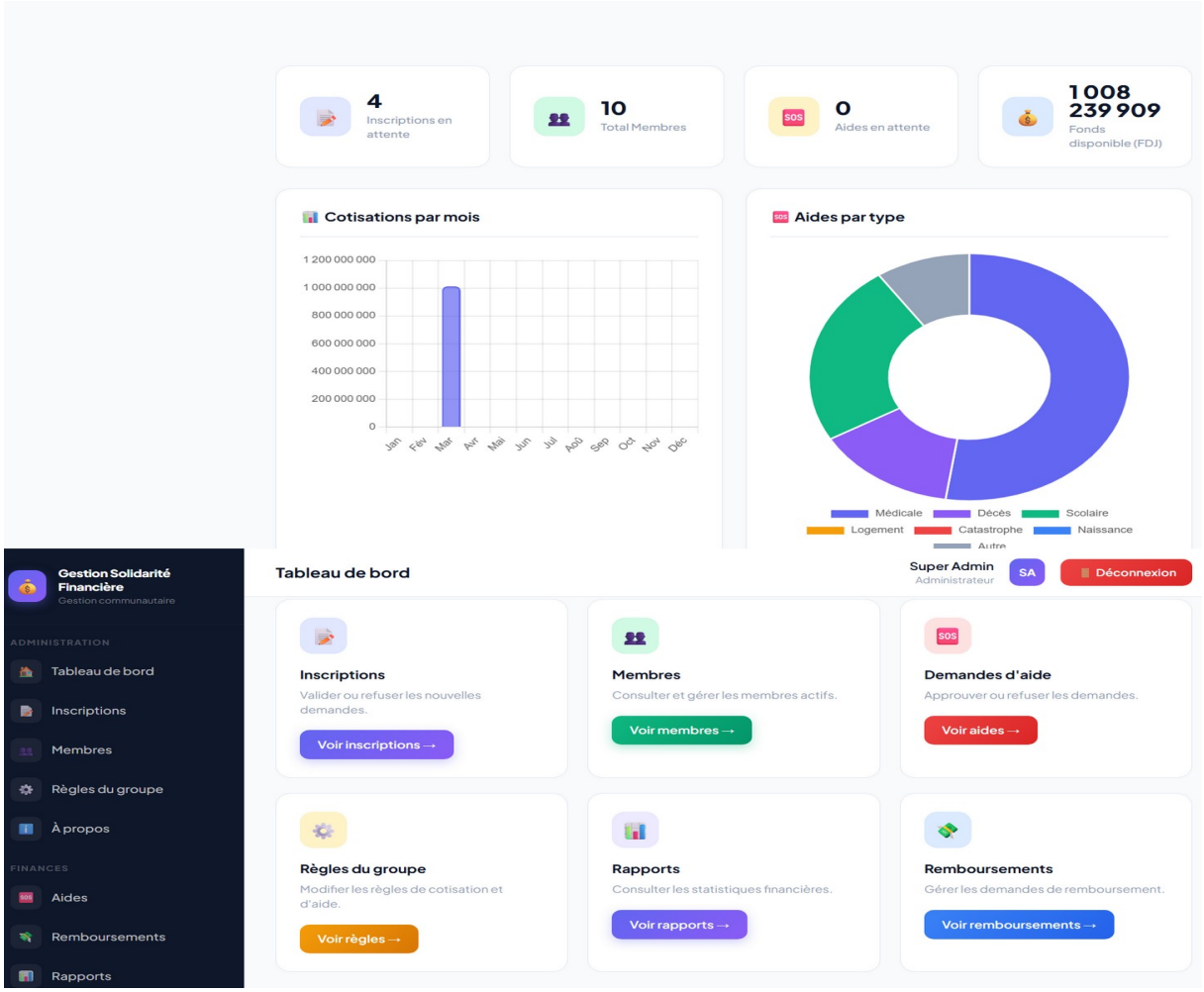
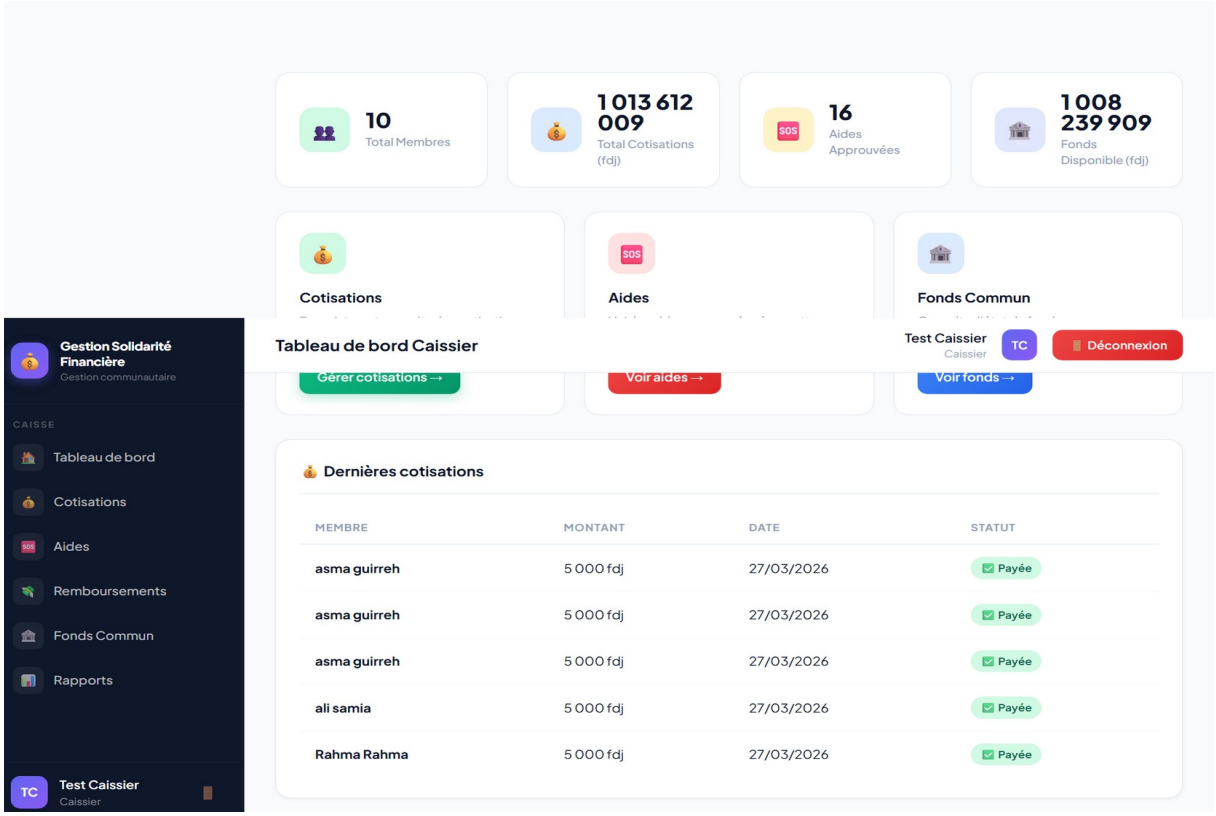


Figure 2 : Tableau de bord (Dashboard)


4
Inscriptions en attente
10
Total Membres
0
Aides en attente
1 008 239 909
Fonds disponible (FDJ)
Cotisations par mois
1 200 000 000
1 000 000 000
800 000 000
600 000 000
400 000 000
200 000 000
0
Jan Fév Mar Avr Mai Jun Jul Aoû Sep Oct Nov Déc
Aides par type
Médicale Décès Scolaire
Logement Catastrophe Naissance
Autre
Gestion Solidarité Financière
Gestion communautaire
ADMINISTRATION
Tableau de bord
Inscriptions
Membres
Règles du groupe
À propos
FINANCES
Aides
Remboursements
Rapports
Tableau de bord
Super Admin
Administrateur
SA
Déconnexion
Inscriptions
Valider ou refuser les nouvelles demandes.
Voir inscriptions →
Membres
Consulter et gérer les membres actifs.
Voir membres →
Demandes d'aide
Approuver ou refuser les demandes.
Voir aides →
Règles du groupe
Modifier les règles de cotisation et d'aide.
Voir règles →
Rapports
Consulter les statistiques financières.
Voir rapports →
Remboursements
Gérer les demandes de remboursement.
Voir remboursements →



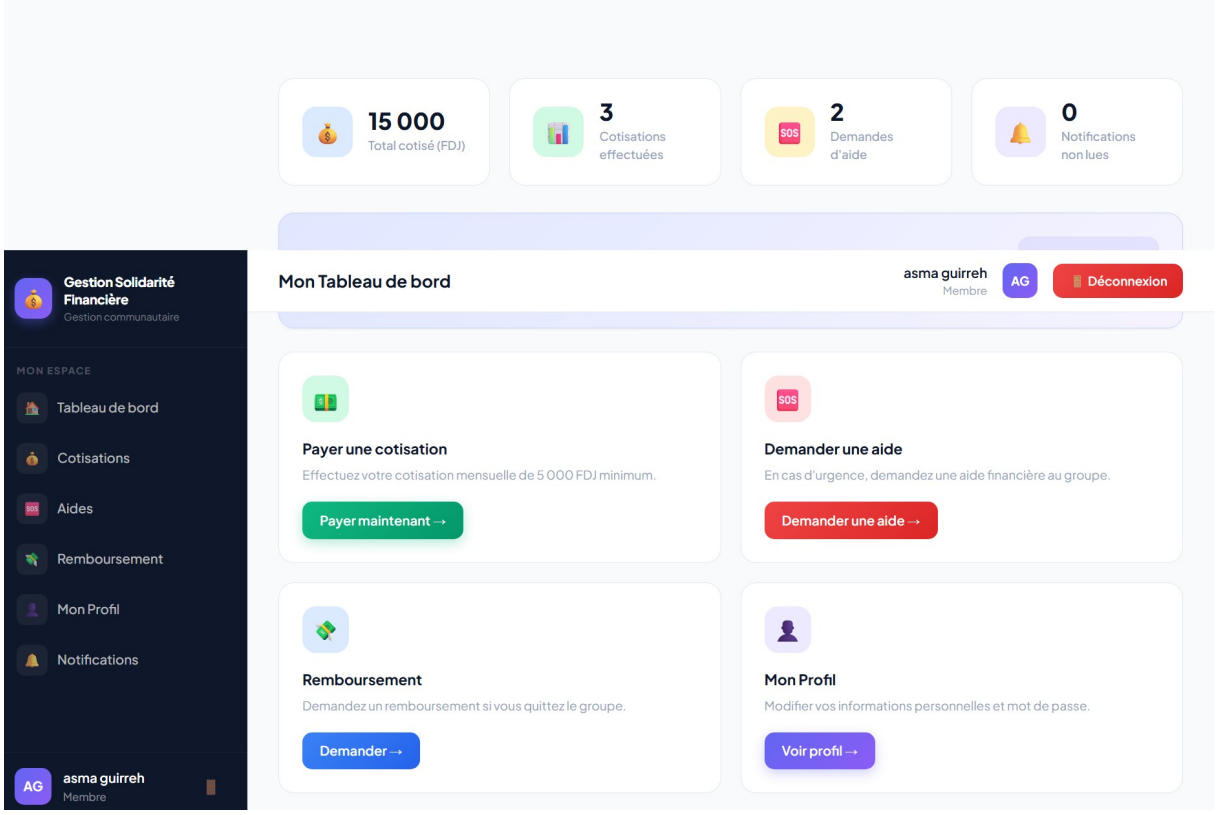
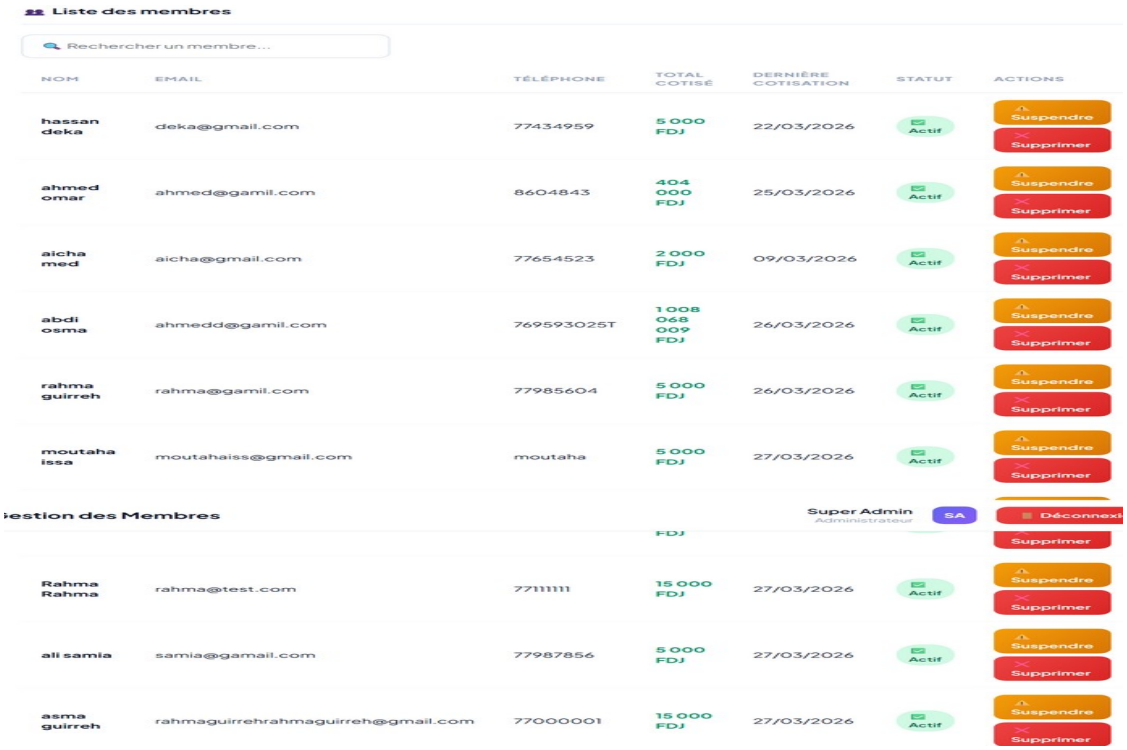


Figure 3 : Liste des membres



Liste des membres

Rechercher un membre...

NOM	EMAIL	TÉLÉPHONE	TOTAL COTISÉ	DERNIÈRE COTISATION	STATUT	ACTIONS
hassan deka	deka@gmail.com	77434959	5 000 FDJ	22/03/2026	Actif	Suspendre Supprimer
ahmed omar	ahmed@gamil.com	8604843	404 000 FDJ	25/03/2026	Actif	Suspendre Supprimer
aicha med	aicha@gmail.com	77654523	2 000 FDJ	09/03/2026	Actif	Suspendre Supprimer
abdi osma	ahmedd@gamil.com	769593025T	1 008 068 009 FDJ	26/03/2026	Actif	Suspendre Supprimer
rahma guirreh	rahma@gamil.com	77985604	5 000 FDJ	26/03/2026	Actif	Suspendre Supprimer
moutaha issa	moutahaiss@gmail.com	moutaha	5 000 FDJ	27/03/2026	Actif	Suspendre Supprimer
			FDJ			Supprimer
Rahma Rahma	rahma@test.com	7711111	15 000 FDJ	27/03/2026	Actif	Suspendre Supprimer
ali samia	samia@gamail.com	77987856	5 000 FDJ	27/03/2026	Actif	Suspendre Supprimer
asma guirreh	rahmaguirrehrahmaguirreh@gmail.com	77000001	15 000 FDJ	27/03/2026	Actif	Suspendre Supprimer

iestion des Membres

Super Admin
Administrateur
SA
Déconnex

Figure 5 : Liste des cotisations

💰 Cotisations

MEMBRE	MONTANT	DATE
asma guirreh	5 000 FDJ	27/03/2026
asma guirreh	5 000 FDJ	27/03/2026
asma guirreh	5 000 FDJ	27/03/2026
ali samia	5 000 FDJ	27/03/2026
Rahma Rahma	5 000 FDJ	27/03/2026
Rahma Rahma	5 000 FDJ	27/03/2026
Rahma Rahma	5 000 FDJ	27/03/2026

Figure 6 : Formulaire d'enregistrement de cotisation

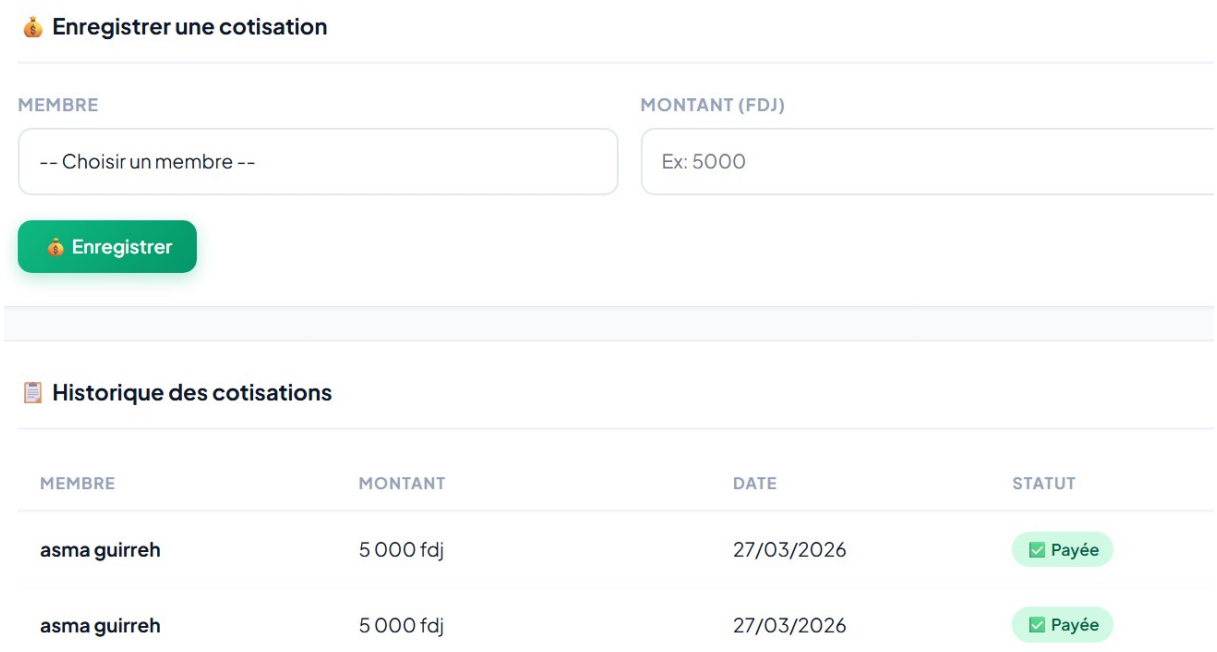


Figure 7 : Demandes d'aide financière

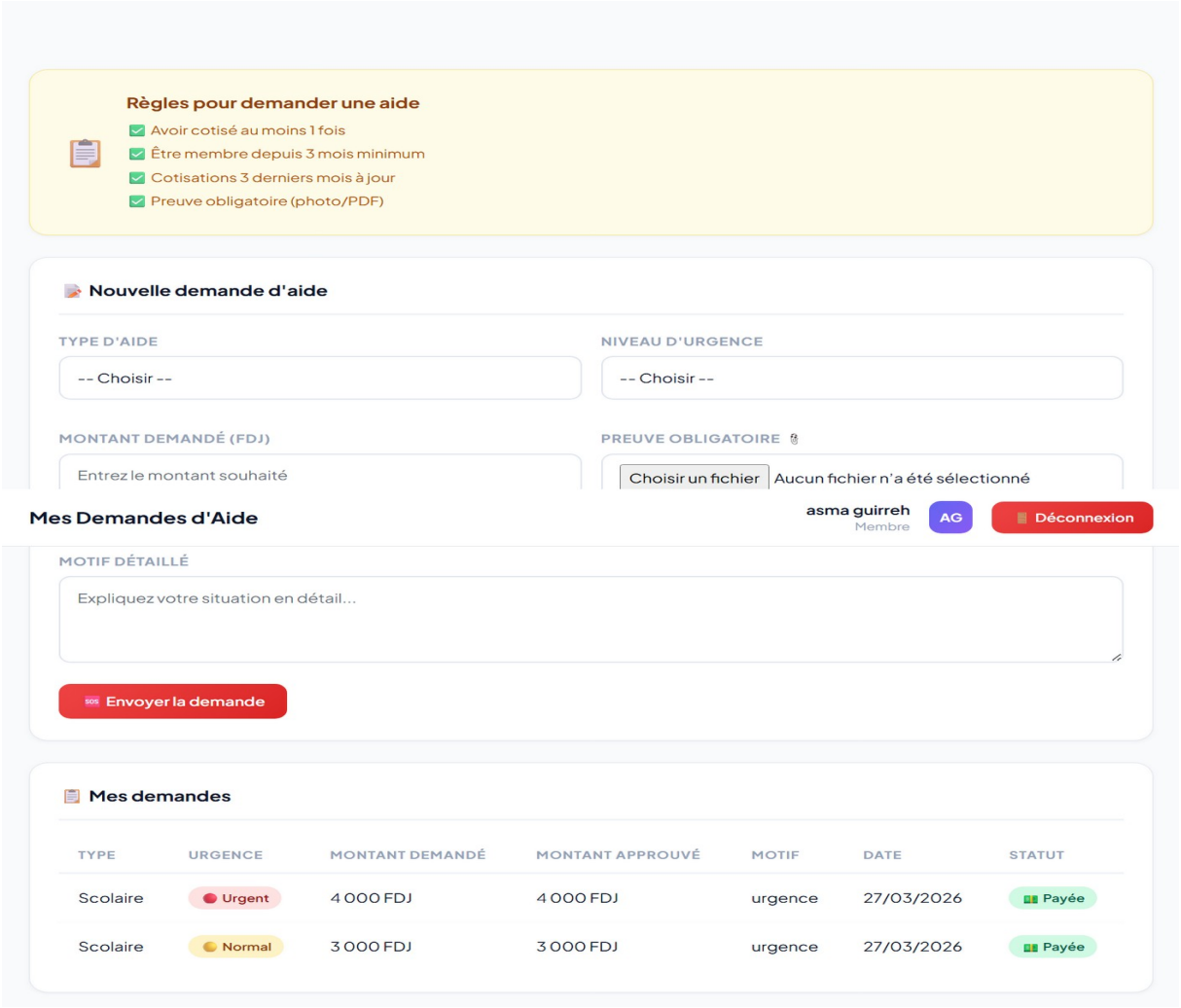
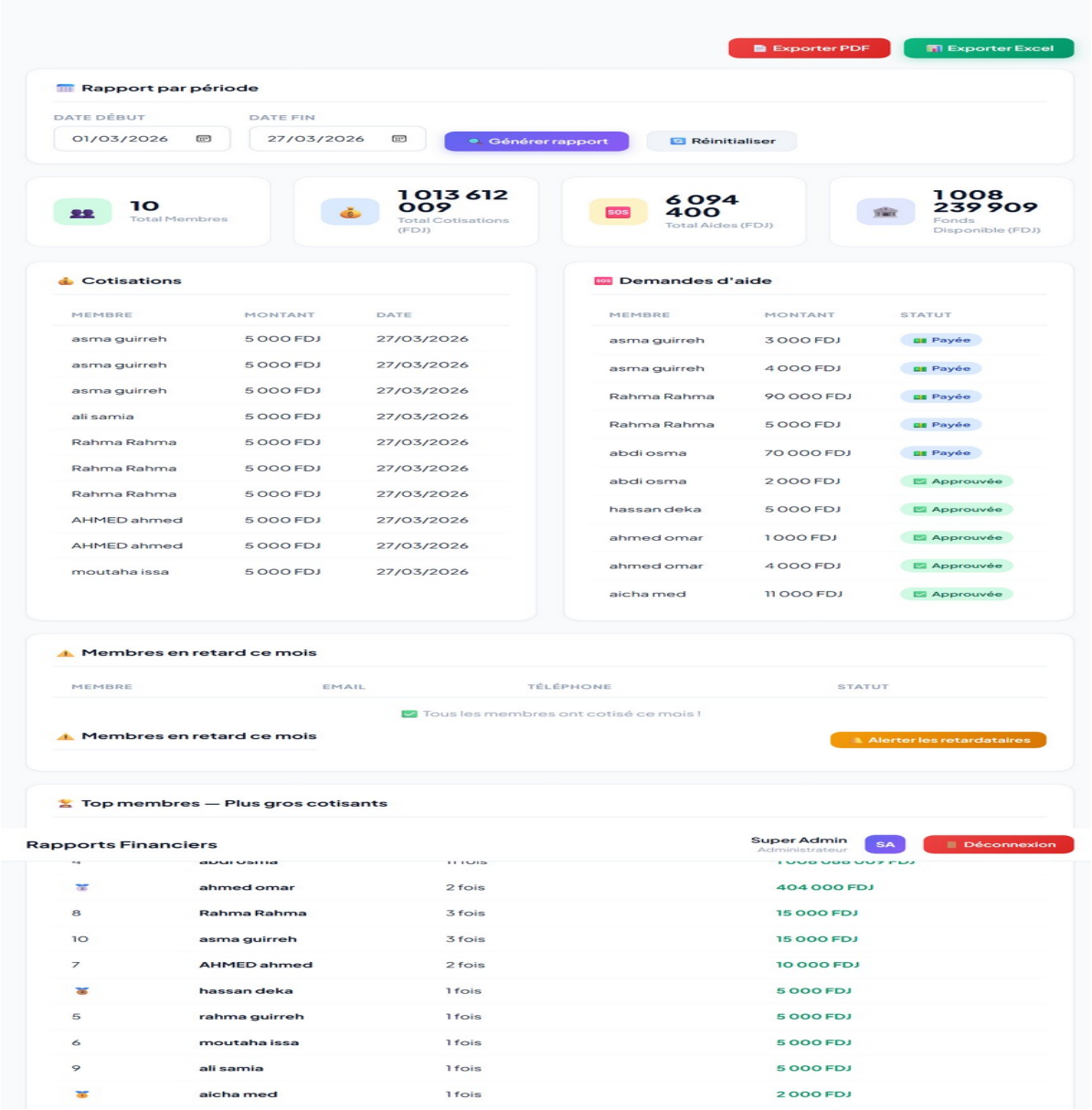


Figure 8 : Rapports financiers



5.2.2 Back-end et base de données (MySQL)

Le back-end de l'application repose entièrement sur le framework Laravel 12, qui offre une structure organisée et sécurisée pour le développement. La base de données utilisée est MySQL, gérée via phpMyAdmin.

Structure de la base de données

La base de données est composée de plusieurs tables principales qui stockent les informations essentielles du système :

Table	Description	Colonnes principales
users	Comptes utilisateurs (admin, caissière, membre)	id, name, email, password, role, created_at
membres	Informations des membres du groupe solidaire	id, nom, prenom, telephone, adresse, date_inscription
cotisations	Enregistrement des cotisations versées	id, membre_id, montant, date_cotisation, statut
aides	Demandes d'aide financière	id, membre_id, montant_demande, motif, statut, date_demande
paiements	Paiements effectués pour les aides approuvées	id, aide_id, montant_paye, date_paiement, caissiere_id
remboursements	Suivi des remboursements des membres	id, membre_id, montant, date_remboursement, statut

Modèles Eloquent

Laravel utilise l'ORM Eloquent pour interagir avec la base de données. Chaque table est représentée par un modèle PHP qui définit les relations entre les entités :

- Membre → hasMany(Cotisation), hasMany(Aide) : un membre peut avoir plusieurs cotisations et plusieurs demandes d'aide ;
- Cotisation → belongsTo(Membre) : chaque cotisation appartient à un membre ;
- Aide → belongsTo(Membre), hasOne(Paiement) : chaque aide est liée à un membre et peut avoir un paiement ;
- Paiement → belongsTo(Aide), belongsTo(User) : chaque paiement est lié à une aide et à la caissière qui l'a effectué ;
- User → belongsTo(Role) : chaque utilisateur a un rôle (Administrateur, Caissière, Membre).

Contrôleurs principaux

Les contrôleurs gèrent la logique métier de l'application. Chaque module dispose de son propre contrôleur qui traite les requêtes HTTP et renvoie les vues correspondantes :

Contrôleur	Rôle
AuthController	Gestion de l'authentification (login, register, logout)
MembreController	CRUD complet des membres (index, create, store, edit, update, destroy)
CotisationController	Enregistrement et consultation des cotisations

AideController	Gestion des demandes d'aide (soumission, validation, refus)
PaiementController	Gestion des paiements par la caissière
DashboardController	Affichage du tableau de bord avec statistiques
RapportController	Génération des rapports financiers

Routes principales

Les routes de l'application sont définies dans le fichier routes/web.php et sont protégées par des middlewares d'authentification et de vérification des rôles :

Méthode	URL	Action
GET	/login	Afficher le formulaire de connexion
POST	/login	Traiter l'authentification
GET	/dashboard	Afficher le tableau de bord
GET	/membres	Lister tous les membres
POST	/membres	Ajouter un nouveau membre
GET	/cotisations	Lister les cotisations
POST	/cotisations	Enregistrer une cotisation
GET	/aides	Lister les demandes d'aide
POST	/aides	Soumettre une demande d'aide
PUT	/aides/{id}/valider	Valider une demande d'aide
GET	/rapports	Afficher les rapports financiers

Sécurité du back-end

Plusieurs mécanismes de sécurité ont été mis en place dans le back-end de l'application :

- Protection CSRF : Laravel génère automatiquement un token CSRF pour chaque formulaire, empêchant les attaques par falsification de requêtes ;
- Hashage des mots de passe : les mots de passe sont chiffrés avec bcrypt avant d'être stockés dans la base de données ;
- Middleware d'authentification : les routes sensibles sont protégées par le middleware 'auth' qui vérifie que l'utilisateur est connecté ;
- Middleware de rôle : un middleware personnalisé vérifie le rôle de l'utilisateur (admin, caissière, membre) avant d'autoriser l'accès ;

- Validation des données : chaque formulaire est validé côté serveur avec les règles de validation Laravel (required, email, numeric, min, max).

5.3 Implémentation du Chat-bot intelligent

En complément des fonctionnalités principales, le système intègre un chat-bot intelligent destiné à assister les utilisateurs dans l'utilisation de la plateforme. Ce chat-bot permet de répondre aux questions fréquentes des membres et de les guider dans leurs démarches.

Objectifs du chat-bot

- Fournir une assistance instantanée aux utilisateurs sans intervention humaine ;
- Répondre aux questions fréquentes (comment payer sa cotisation, comment faire une demande d'aide, etc.) ;
- Guider les nouveaux membres dans la prise en main de la plateforme ;
- Réduire la charge de travail de l'administrateur en automatisant les réponses aux questions courantes.

Fonctionnement technique

Le chat-bot fonctionne selon un système de questions-réponses prédéfinies. Il analyse les mots-clés saisis par l'utilisateur et renvoie la réponse la plus pertinente. L'implémentation repose sur les éléments suivants :

- Interface de chat : une fenêtre de conversation intégrée en bas à droite de l'écran, accessible depuis toutes les pages ;
- Traitement des messages : un contrôleur Laravel (ChatbotController) reçoit le message de l'utilisateur et recherche la réponse appropriée ;
- Base de connaissances : une table 'chatbot_responses' dans MySQL stocke les questions fréquentes et leurs réponses ;
- Recherche par mots-clés : le système compare les mots-clés du message avec ceux enregistrés dans la base de connaissances ;
- Réponse par défaut : si aucune correspondance n'est trouvée, le chat-bot propose de contacter l'administrateur.

Exemples de questions gérées

Question de l'utilisateur	Réponse du chat-bot
Comment payer ma cotisation ?	Rendez-vous dans la section 'Cotisations' et cliquez sur 'Nouvelle cotisation'. Sélectionnez votre nom et entrez le montant.
Comment faire une demande d'aide ?	Allez dans 'Demandes d'aide', cliquez sur

	'Nouvelle demande', remplissez le motif et le montant souhaité.
Quel est mon solde ?	Consultez votre tableau de bord pour voir le résumé de vos cotisations et aides reçues.
Comment contacter l'administrateur ?	Vous pouvez contacter l'administrateur via la section 'Contact' ou par email.
Comment modifier mes informations ?	Rendez-vous dans 'Mon profil' pour modifier vos informations personnelles.

Le chat-bot constitue une fonctionnalité complémentaire qui améliore l'expérience utilisateur et rend le système plus accessible, notamment pour les membres qui découvrent la plateforme pour la première fois.

CHAPITRE 6 : TESTS

6.1 Stratégie de tests (Unitaires, Intégration, Système, Acceptation(recette))

1. Test Unitaire

Le test unitaire consiste à tester chaque fonctionnalité de manière individuelle et isolée pour s'assurer qu'elle produit le bon résultat.
Dans notre projet, nous avons commencé par tester la fonction d'inscription des membres pour vérifier que les informations sont bien enregistrées. Ensuite nous avons testé la fonction de calcul des cotisations pour s'assurer que les montants sont calculés correctement. Nous avons également testé la fonction de versement de fonds pour vérifier que les opérations financières sont bien enregistrées. Enfin nous avons testé la fonction de connexion pour s'assurer que seuls les utilisateurs autorisés peuvent accéder à l'application.

1. Test d'Intégration

Le test d'intégration permet de vérifier que les différents modules de l'application fonctionnent correctement ensemble.
Nous avons d'abord vérifié la communication entre le module membre et la base de données. Cela nous a permis de confirmer que les informations des membres sont bien sauvegardées et récupérées correctement. Ensuite nous avons testé la liaison entre le module de paiement et le module d'historique pour vérifier que chaque transaction effectuée apparaît automatiquement dans l'historique. Nous avons également vérifié que le module d'authentification reconnaît bien les rôles des utilisateurs, c'est-à-dire que l'administrateur accède aux fonctions d'administration et que le membre simple n'accède qu'à ses propres informations.

2. Test Système

Le test système consiste à tester l'application dans sa totalité en simulant des situations réelles d'utilisation.
Nous avons simulé le parcours complet d'un nouveau membre, depuis son inscription jusqu'à la consultation de son historique de transactions. Nous avons également simulé le processus de versement de fonds, en partant de la demande du membre jusqu'à la validation par l'administrateur et la mise à jour du solde. Enfin nous avons testé la génération des rapports financiers pour vérifier que les données affichées sont correctes et exportables.

3. Test d'Acceptation

Le test d'acceptation est la dernière étape de validation. Il permet de confirmer que l'application répond bien aux besoins réels pour lesquels elle a été conçue.

Nous avons vérifié que l'interface est simple et facile à utiliser pour tous les membres, même ceux qui ne sont pas habitués à l'informatique. Nous avons confirmé que les calculs financiers sont exacts et que les rapports générés reflètent bien la réalité des opérations effectuées. Nous avons également vérifié que les données des membres et leurs informations financières sont bien protégées et sécurisées.

6.1.1 Scénarios de test détaillés

Cette section présente l'ensemble des scénarios de test réalisés pour valider le bon fonctionnement de chaque module de l'application de gestion de solidarité financière.

Scénario 1 : Authentification

N°	Action	Résultat attendu	Statut
1.1	Accéder à la page de connexion	Le formulaire de connexion s'affiche correctement	Validé
1.2	Se connecter avec un email et mot de passe valides	Redirection vers le tableau de bord selon le rôle	Validé
1.3	Se connecter avec un mot de passe incorrect	Message d'erreur : 'Identifiants incorrects'	Validé
1.4	Se connecter avec un email non enregistré	Message d'erreur : 'Aucun compte trouvé'	Validé
1.5	Cliquer sur 'Déconnexion'	Redirection vers la page de connexion, session détruite	Validé
1.6	Accéder au dashboard sans être connecté	Redirection automatique vers la page de connexion	Validé

Scénario 2 : Gestion des membres

N°	Action	Résultat attendu	Statut
2.1	Afficher la liste des membres	Tous les membres s'affichent dans un	Validé

		tableau	
2.2	Ajouter un nouveau membre avec des données valides	Le membre est enregistré et apparaît dans la liste	Validé
2.3	Ajouter un membre sans remplir les champs obligatoires	Message d'erreur de validation affiché	Validé
2.4	Modifier les informations d'un membre	Les modifications sont sauvegardées correctement	Validé
2.5	Supprimer un membre	Le membre est supprimé après confirmation	Validé
2.6	Rechercher un membre par nom	Le membre correspondant s'affiche	Validé

Scénario 3 : Gestion des cotisations

N°	Action	Résultat attendu	Statut
3.1	Afficher la liste des cotisations	Toutes les cotisations s'affichent avec le nom du membre	Validé
3.2	Enregistrer une nouvelle cotisation	La cotisation est enregistrée avec la date et le montant	Validé
3.3	Enregistrer une cotisation avec montant invalide (0 ou négatif)	Message d'erreur de validation	Validé
3.4	Consulter l'historique des cotisations d'un membre	Les cotisations du membre sélectionné s'affichent	Validé
3.5	Vérifier le calcul du total des cotisations	Le total affiché correspond à la somme des cotisations	Validé

Scénario 4 : Gestion des demandes d'aide financière

N°	Action	Résultat attendu	Statut
4.1	Soumettre une	La demande est	Validé

	demande d'aide avec motif et montant	enregistrée avec le statut 'En attente'	
4.2	Soumettre une demande sans motif	Message d'erreur : le motif est obligatoire	Validé
4.3	L'administrateur valide une demande	Le statut passe à 'Approuvée'	Validé
4.4	L'administrateur refuse une demande	Le statut passe à 'Refusée'	Validé
4.5	La caissière effectue le paiement d'une aide approuvée	Le paiement est enregistré, le solde est mis à jour	Validé
4.6	Consulter l'historique des demandes d'aide d'un membre	Toutes les demandes s'affichent avec leur statut	Validé

Scénario 5 : Tableau de bord et statistiques

N°	Action	Résultat attendu	Statut
5.1	Accéder au tableau de bord (Admin)	Les statistiques globales s'affichent (membres, cotisations, solde)	Validé
5.2	Vérifier le nombre total de membres	Le nombre affiché correspond au nombre réel dans la BDD	Validé
5.3	Vérifier le total des cotisations	Le montant affiché correspond à la somme des cotisations	Validé
5.4	Vérifier le solde de la caisse	Solde = Total cotisations - Total aides payées	Validé
5.5	Accéder au tableau de bord (Caissière)	Seules les informations financières sont visibles	Validé

Scénario 6 : Génération des rapports

N°	Action	Résultat attendu	Statut
6.1	Générer un rapport	Le rapport s'affiche	Validé

	des cotisations	avec détails par membre et par date	
6.2	Générer un rapport des dépenses (aides payées)	Le rapport liste toutes les aides payées	Validé
6.3	Afficher les statistiques globales	Résumé complet : entrées, sorties, solde	Validé

Scénario 7 : Sécurité et contrôle d'accès

N°	Action	Résultat attendu	Statut
7.1	Un membre tente d'accéder à la page d'administration	Accès refusé, redirection vers sa page d'accueil	Validé
7.2	Un utilisateur non connecté tente d'accéder aux données	Redirection vers la page de connexion	Validé
7.3	Vérifier le chiffrement des mots de passe en BDD	Les mots de passe sont stockés en bcrypt (hashés)	Validé
7.4	Soumettre un formulaire sans token CSRF	Erreur 419 : Page expirée	Validé
7.5	Tenter une injection SQL dans un champ de recherche	La requête est sécurisée, aucune injection possible	Validé

Scénario 8 : Chat-bot intelligent

N°	Action	Résultat attendu	Statut
8.1	Ouvrir la fenêtre du chat-bot	La fenêtre de chat s'affiche en bas à droite	Validé
8.2	Poser une question connue ('comment payer ?')	Le chat-bot retourne la réponse appropriée	Validé
8.3	Poser une question inconnue	Le chat-bot propose de contacter l'administrateur	Validé
8.4	Fermer la fenêtre du	La fenêtre se ferme	Validé

	chat-bot	correctement

Résumé des résultats de test

Scénario	Nombre de tests	Tests validés	Tests échoués
Authentification	6	6	0
Gestion des membres	6	6	0
Gestion des cotisations	5	5	0
Demandes d'aide financière	6	6	0
Tableau de bord	5	5	0
Rapports	3	3	0
Sécurité et contrôle d'accès	5	5	0
Chat-bot intelligent	4	4	0
TOTAL	40	40	0

L'ensemble des 40 scénarios de test ont été validés avec succès, ce qui confirme le bon fonctionnement de tous les modules de l'application de gestion de solidarité financière.

6.2 Évaluation des performances

L'évaluation des performances vise à mesurer la capacité du système à répondre efficacement aux demandes des utilisateurs, même lorsque le volume de données ou le nombre d'opérations augmente. Dans le cas de notre système de gestion de solidarité financière, cette évaluation est importante car l'application doit pouvoir traiter rapidement les opérations courantes, telles que la consultation des membres, l'enregistrement des cotisations et l'accès aux historiques de paiement.

Plusieurs critères ont été considérés dans cette évaluation :

- le temps de chargement des pages ;
- le temps de réponse lors de l'enregistrement d'une cotisation ;
- la rapidité d'affichage des listes de membres ou des opérations ;
- la fluidité de navigation entre les différentes interfaces ;
- la capacité du système à supporter plusieurs connexions simultanées.

Grâce à Laravel 12 et à MySQL, le système bénéficie d'une architecture moderne et performante. L'utilisation d'un schéma relationnel bien structuré, l'optimisation des requêtes

SQL, l'indexation de certaines colonnes importantes (comme id_membre, date_cotisation, statut_paiement) et une bonne gestion des relations Eloquent entre les tables (Membre, Cotisation, Aide, Remboursement) permettent d'améliorer les performances générales de l'application.

Dans le cadre de ce projet, l'évaluation des performances a montré que l'application reste rapide pour les opérations essentielles, ce qui garantit une bonne expérience utilisateur aussi bien pour la caissière que pour l'administrateur et les membres.

CHAPITRE 7 : Déploiement et Maintenance

7.1 Déploiement

7.1.1 Architecture de déploiement

L'architecture de déploiement du système repose sur une organisation classique en trois niveaux :

la couche présentation, représentée par les interfaces web accessibles aux membres, à la caissière et à l'administrateur ;

la couche logique applicative, gérée par Laravel 12, qui traite les règles métiers, les accès utilisateurs, les validations et les opérations de gestion ;

la couche données, assurée par la base de données MySQL, qui stocke les informations relatives aux membres, cotisations, paiements, utilisateurs et historiques.

Dans cette architecture, les utilisateurs accèdent à l'application à travers un navigateur web. Les requêtes sont envoyées au serveur Laravel, qui traite les demandes puis communique avec la base de données MySQL pour lire, enregistrer ou modifier les informations. Cette architecture permet une séparation claire des responsabilités, facilite la maintenance et améliore la sécurité du système.

7.1.2 Option de déploiement (Déploiement local, Déploiement web)

7.1.2 Options de déploiement

Le système peut être déployé selon deux principales options : le déploiement local et le déploiement web.

Déploiement local

Le déploiement local consiste à installer l'application sur une machine ou sur un réseau local, généralement dans l'environnement de développement ou de test. Cette solution est utile pendant la phase de conception, de test et de validation interne. Elle permet aux développeurs d'exécuter Laravel 12 localement, par exemple avec un serveur Apache ou Nginx, tout en utilisant MySQL comme système de gestion de base de données.

Cette option présente plusieurs avantages :

simplicité de mise en place ;

rapidité pour les tests ;

contrôle total de l'environnement ;

absence de dépendance à Internet.

Cependant, le déploiement local reste limité, car l'application n'est accessible qu'à partir des machines autorisées sur le réseau concerné.

Déploiement web

Le déploiement web consiste à héberger l'application sur un serveur distant afin qu'elle soit accessible en ligne via un navigateur. Cette solution est la plus adaptée si l'on souhaite rendre le système disponible à plusieurs utilisateurs depuis différents lieux.

Dans ce cas, le projet Laravel 12 est installé sur un hébergement compatible PHP et MySQL. Le système peut alors être utilisé à distance par les membres, la caissière et l'administrateur, sous réserve d'une authentification sécurisée. Ce mode de déploiement favorise l'accessibilité, la centralisation des données et la gestion en temps réel.

Le déploiement web nécessite néanmoins certaines précautions, notamment au niveau de la sécurité, des sauvegardes et de la disponibilité du serveur.

7.1.3 Sécurisation du déploiement

La sécurisation du déploiement est indispensable pour protéger les données financières et personnelles manipulées par le système. En effet, l'application contient des informations sensibles, comme les identités des membres, les montants des cotisations, les historiques de paiement et les comptes utilisateurs.

Plusieurs mesures de sécurité doivent être mises en place :

authentification sécurisée des utilisateurs

mots de passe chiffrés dans la base de données

séparation des accès selon les rôles : membre, caissière, administrateur

validation stricte des données saisies

protection contre les accès non autorisés

sauvegarde régulière de la base de données

utilisation d'un certificat SSL en cas de déploiement web pour sécuriser les échanges.

Laravel 12 fournit naturellement plusieurs mécanismes utiles, tels que la protection CSRF, le hashage des mots de passe, la gestion des sessions et des middlewares d'autorisation. Ces outils renforcent la sécurité globale du système et réduisent les risques de fraude, de perte de données ou de compromission des comptes utilisateurs.

7.2 Maintenance

7.2.1 Maintenance correction.

La maintenance corrective consiste à corriger les erreurs ou anomalies détectées après la mise en production du système. Ces erreurs peuvent être liées à des bugs dans le code, des problèmes d'affichage ou des dysfonctionnements dans certaines fonctionnalités.

Dans le cadre du système de gestion de solidarité financière, cette maintenance permet par exemple de corriger une erreur dans le calcul des cotisations, un problème lors de l'enregistrement des transactions ou un bug empêchant la connexion des utilisateurs.

L'objectif principal de la maintenance corrective est d'assurer le bon fonctionnement du système et de garantir la fiabilité des données.

7.2.1 Maintenance evolutive

La maintenance évolutive consiste à améliorer le système en ajoutant de nouvelles fonctionnalités ou en optimisant celles déjà existantes afin de répondre aux nouveaux besoins des utilisateurs.

Dans notre projet, cela peut inclure l'ajout d'un module de paiement en ligne, l'intégration d'un système de notification par SMS ou email, ou encore l'amélioration de l'interface utilisateur pour la rendre plus intuitive.

Cette maintenance permet de faire évoluer l'application dans le temps et de l'adapter aux attentes des utilisateurs et aux nouvelles technologies.

7.2.3 Maintenance adaptative.

La maintenance adaptative consiste à modifier le système pour qu'il reste compatible avec son environnement technique en constante évolution.

Cela inclut par exemple la mise à jour du système pour qu'il fonctionne avec une nouvelle version de PHP, de Laravel ou du système de gestion de base de données MySQL. Elle peut également concerner l'adaptation à de nouveaux systèmes d'exploitation ou navigateurs.

L'objectif de la maintenance adaptative est de garantir la continuité du fonctionnement du système malgré les changements technologiques.

Conclusion

Au terme de ce projet, nous avons conçu et réalisé une application de gestion de solidarité financière, permettant de centraliser et de suivre efficacement les cotisations et les aides financières au sein d'un groupe ou d'une organisation.

Ce travail nous a permis de répondre à la problématique posée, en proposant une solution facilitant la gestion des membres, le suivi des fonds, le traitement des demandes d'aide et la génération de rapports financiers fiables. L'application développée intègre plusieurs fonctionnalités essentielles telles que la gestion des utilisateurs, le calcul et l'enregistrement des cotisations, le suivi des aides et l'affichage d'indicateurs financiers pour la prise de décision.

Tout au long de ce projet, nous avons mobilisé nos compétences en analyse, en modélisation UML, en conception de base de données et en développement informatique. Nous avons également appris à organiser les tâches, à travailler de manière méthodique et à surmonter les difficultés techniques rencontrées.

Malgré certaines contraintes liées au temps et à la complexité de certaines fonctionnalités, les objectifs fixés ont été atteints. Des améliorations futures peuvent être envisagées, comme l'intégration de services de paiement en ligne, le développement d'une application mobile ou l'ajout de fonctionnalités avancées de notification et de reporting.

En conclusion, ce projet a constitué une expérience enrichissante qui nous a permis de développer nos compétences techniques et organisationnelles, tout en proposant une solution utile et évolutive pour la gestion de la solidarité financière.

BIBLIOGRAPHIE

Ouvrages :

- Sommerville, I. (2016). Software Engineering (10th Edition). Pearson.
- Pressman, R. S. (2014). Software Engineering: A Practitioner’s Approach. McGraw-Hill.

Sites Web :

- https://laravel.com/docs
- https://www.php.net
- https://www.mysql.com
- https://www.w3schools.com

Documents et supports de cours :

- Cours de génie logiciel – Université de Djibouti
- Cours de bases de données – Université de Djibouti
- Cours de développement web (PHP, Laravel)

Outils et technologies utilisés :

- Laravel 12 (Framework PHP)
- MySQL (Système de gestion de base de données)
- Visual Studio Code (Éditeur de code)
- Git (Gestion de version)